
Fundamentals – Distributed Algorithms

Coordination and Agreement

Introduction

- How do processes coordinate their actions and agree on shared values in distributed systems?
 - e.g., among a set of computers, one is selected as a designated time server with which the remaining computers synchronize their clocks – if this server fails, how to choose another computer to take over?
- Problems and issues
 - **Mutual exclusion** – for processes to coordinate their accesses to shared resources
 - **Election** – for processes to agree on a coordinator
 - **Consensus** – for processes to agree on some value

Assumptions

- Each pair of processes is connected by reliable channels – any message sent is eventually delivered intact, exactly once
- No process failure implies a threat to the other processes' ability to communicate – processes do not depend on one another to forward messages
- Communication channels between processes are private
 - A process does not know the messages sent over channels irrelevant to it
 - A process cannot inject messages into the channel between a pair of other processes

Outline

- Distributed Mutual Exclusion
- Election
- Consensus Problem

Mutual Exclusion

- Objective

- To prevent interference when a set of processes access shared resources that require exclusive use, e.g., printers, databases, data structures
- An extension to the critical section problem in operating systems

- Distributed mutual exclusion

- Unlike stand-alone systems, in a distributed system, neither shared variables nor facilities supplied by a single local kernel can solve the problem
- So, we require a solution that is based solely on **message passing**

System Model

- A collection of N processes $p_1, p_2, \dots, p_N$ that do not share variables
- The processes access shared resources in a critical section
 - For simplicity, assume only one critical section
 - Application-level protocol for a critical section
 - enter() //enter critical section – block if necessary
 - resourceAccess() //access shared resources in critical section
 - exit() //leave critical section – other processes may now enter
- Asynchronous system (delays are unbounded)

Requirements for Mutual Exclusion

- Safety

- **At most one process** may execute in the critical section at a time

- Liveness

- Requests to enter and exit the critical section eventually succeed
- **→ no deadlock** (deadlock: several processes are waiting for each other to proceed)
- **→ no starvation** (starvation: a requesting process is infinitely postponed for entry to the critical section)

Performance Metrics

- **Bandwidth consumption**
 - Number of messages sent in each entry and exit operation
- **Client delay**
 - Delay incurred at a process at each entry operation
 - When the time accessing the resources in the critical section is short, the dominant factor in the delay is the actual mechanism for mutual exclusion
 - When the resources are used for a long time period in the critical section, the dominant factor is waiting for everyone else to take their turn
 - We focus on the former case

Performance Metrics

■ Throughput

- Rate at which the set of processes as a whole can access the critical section
- **Synchronization delay** – between one process exiting the critical section and the next process entering it
- Shorter synchronization delay → higher throughput
- We focus on the synchronization delay when every process constantly wants to enter a critical section

An Example

- An example to use throughout this section

- A set of 9 processes $p_1, p_2, \dots, p_9$
- They require entry to the critical section at the following physical times:

p_1	1 sec
p_6	2 sec
p_2	3 sec
p_8	5 sec

- Once entering the critical section, a process accesses the shared resource for 3 seconds before exiting the critical section
- For simplicity, we ignore message transmission delay in the network and message processing delay at the processes in the examples

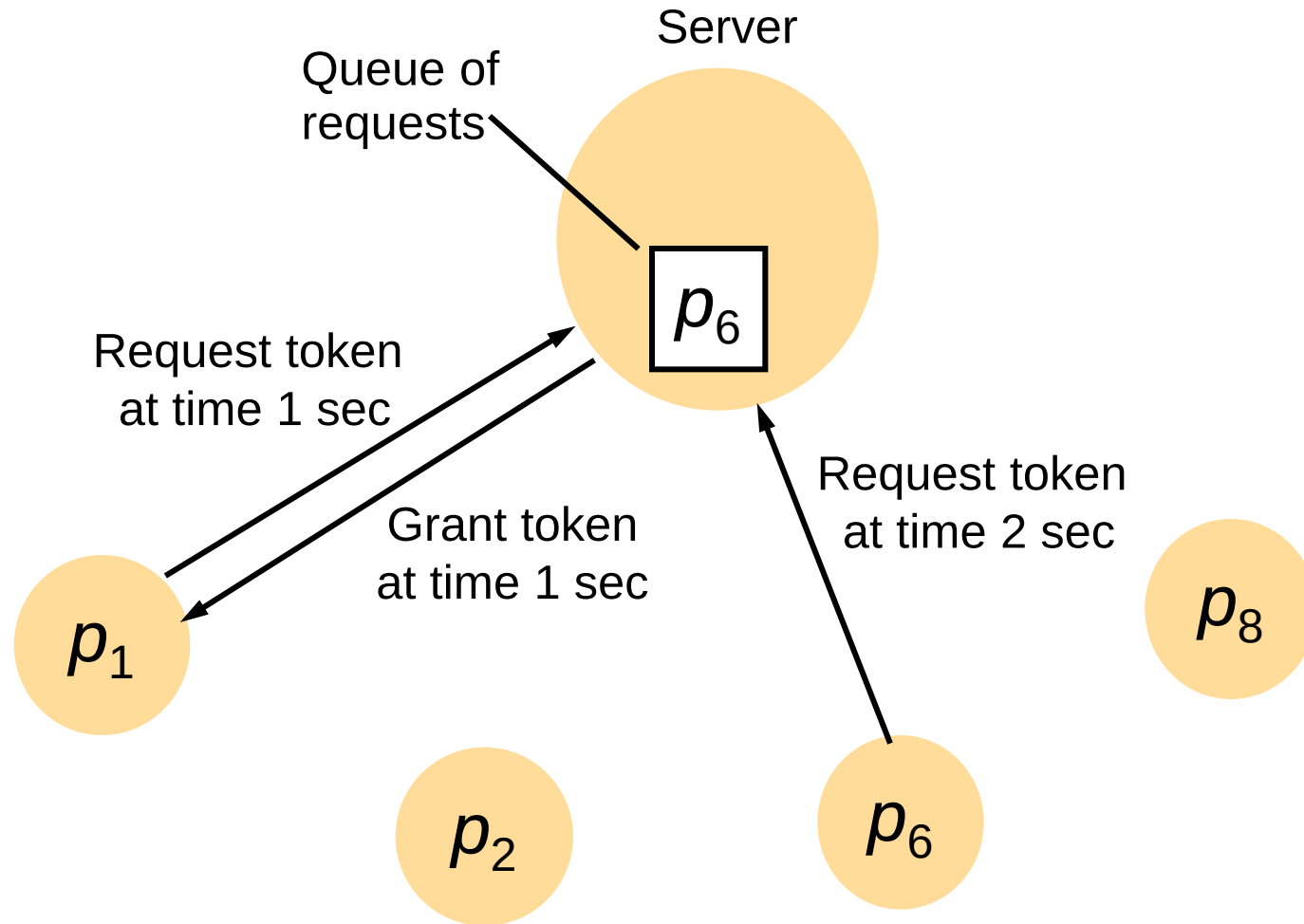
Central Server Algorithm

- Employ a server that grants permission to enter the critical section
 - The server can be an **external process** or **one of the N processes** $p_1, p_2, \dots, p_N$ to be coordinated
- To enter a critical section
 - A process sends a request to the server and awaits a reply from it
 - **Token** – permission to enter the critical section
 - If no other process has the token, the server replies immediately, granting the token, on receiving which the requesting process enters the critical section
 - Otherwise, the server queues the request

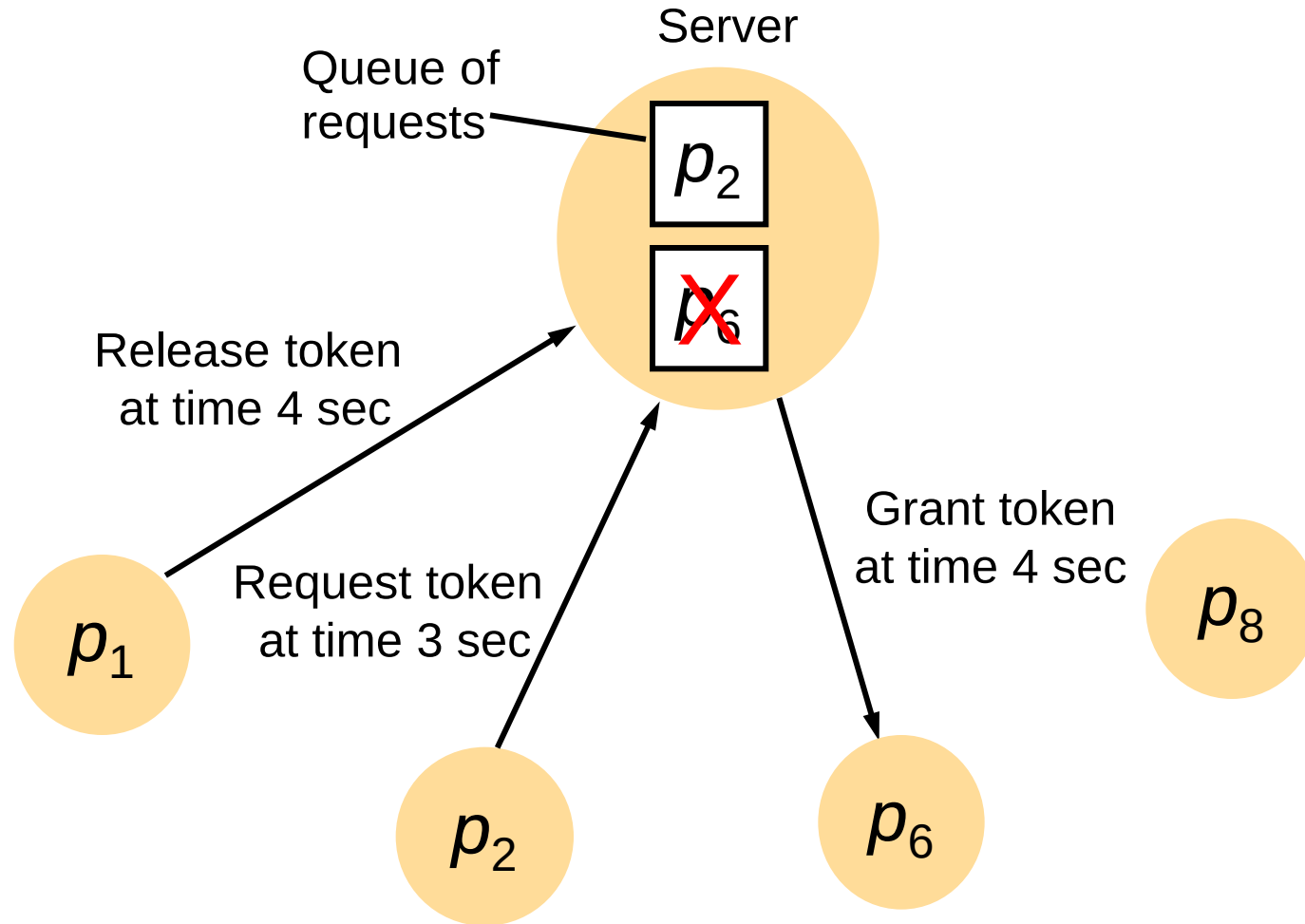
Central Server Algorithm

- To exit a critical section
 - A process sends the token back to the server
 - If the waiting queue is not empty, the server chooses the oldest request in the queue, removes it and replies to the corresponding process with the token

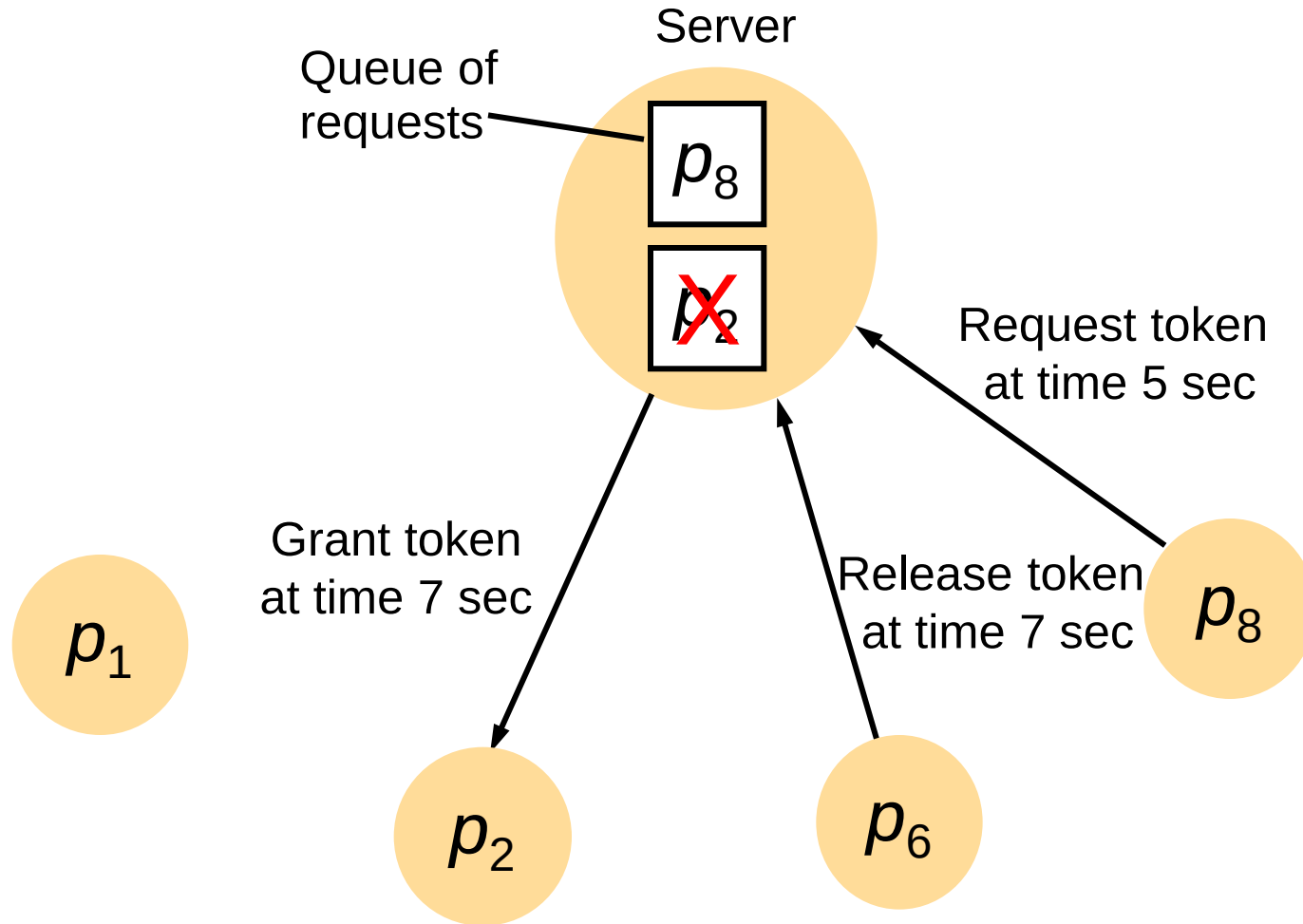
Example of Central Server Algorithm



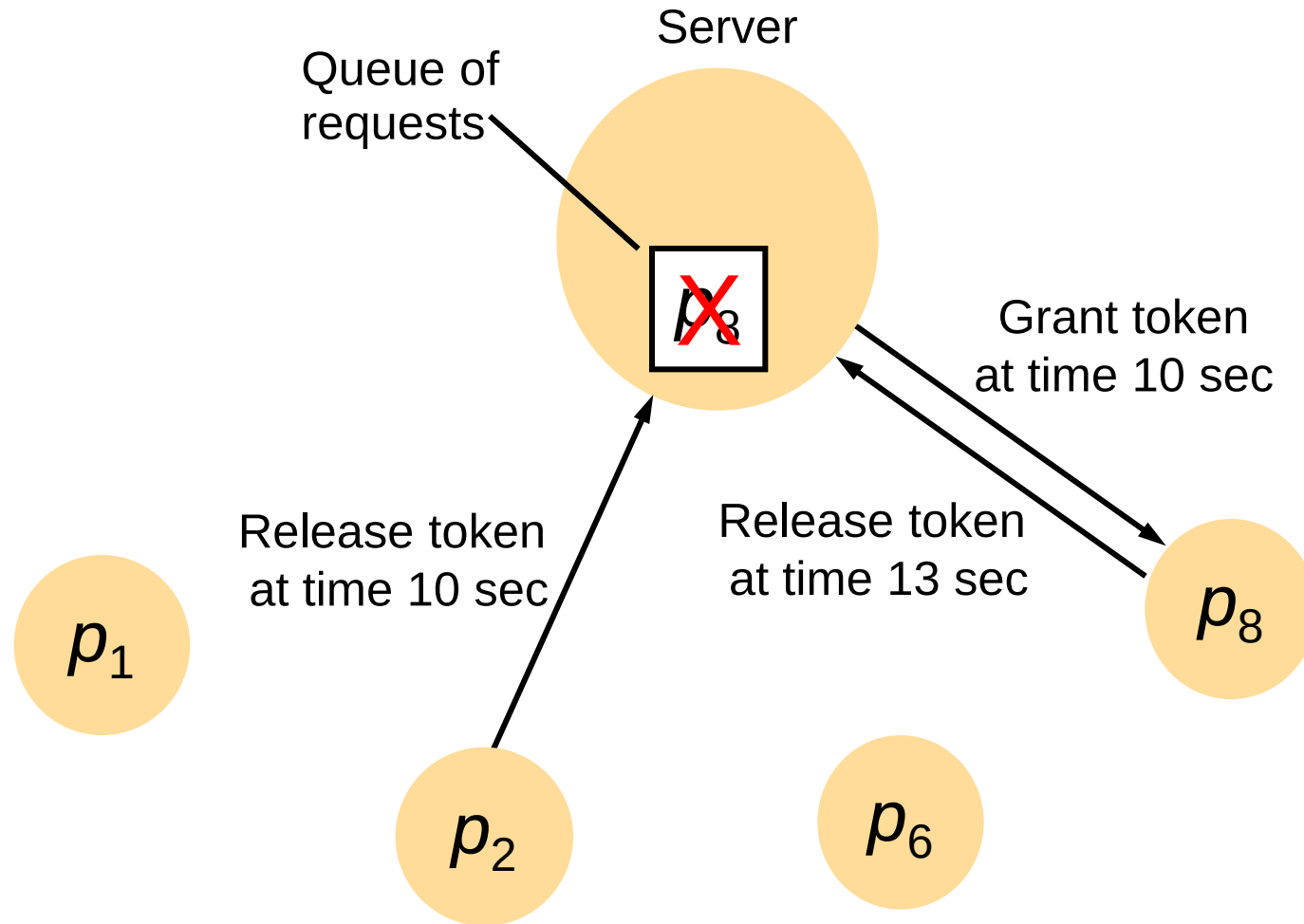
Example of Central Server Algorithm



Example of Central Server Algorithm



Example of Central Server Algorithm



Analysis of Central Server Algorithm

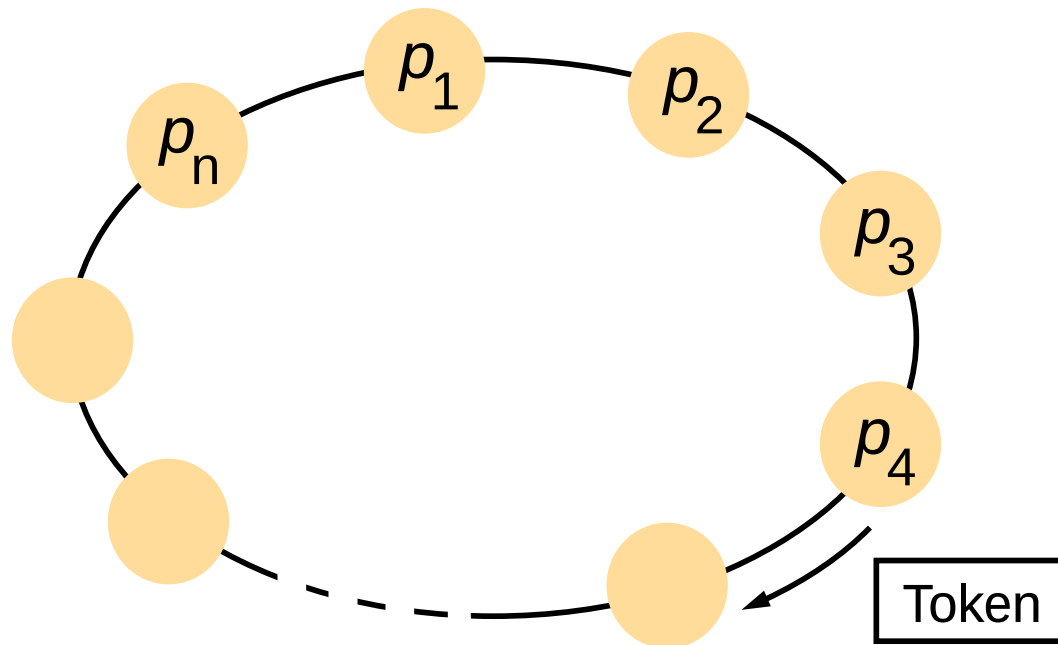
- Safety – satisfied since there is only one token
- Liveness – satisfied due to FIFO request queue
 - Requests are granted in the order in which they are received by the central server
 - No process waits forever → no starvation
- Disadvantage – the server is a single point of failure and can become a performance bottleneck

Analysis of Central Server Algorithm

- Bandwidth consumption
 - Entering a critical section takes **two messages** (a request + a grant)
 - Exiting a critical section takes **one message** (release)
- Client delay
 - Entering a critical section delays the process by **a round-trip time**
- Synchronization delay
 - **A round-trip time** (a release message to the server followed by a grant message to the next process to enter the critical section)

Ring-Based Algorithm

- How to achieve mutual exclusion without a central server process?
- Arrange processes in **a logical ring**
 - Each process knows the next process around the ring

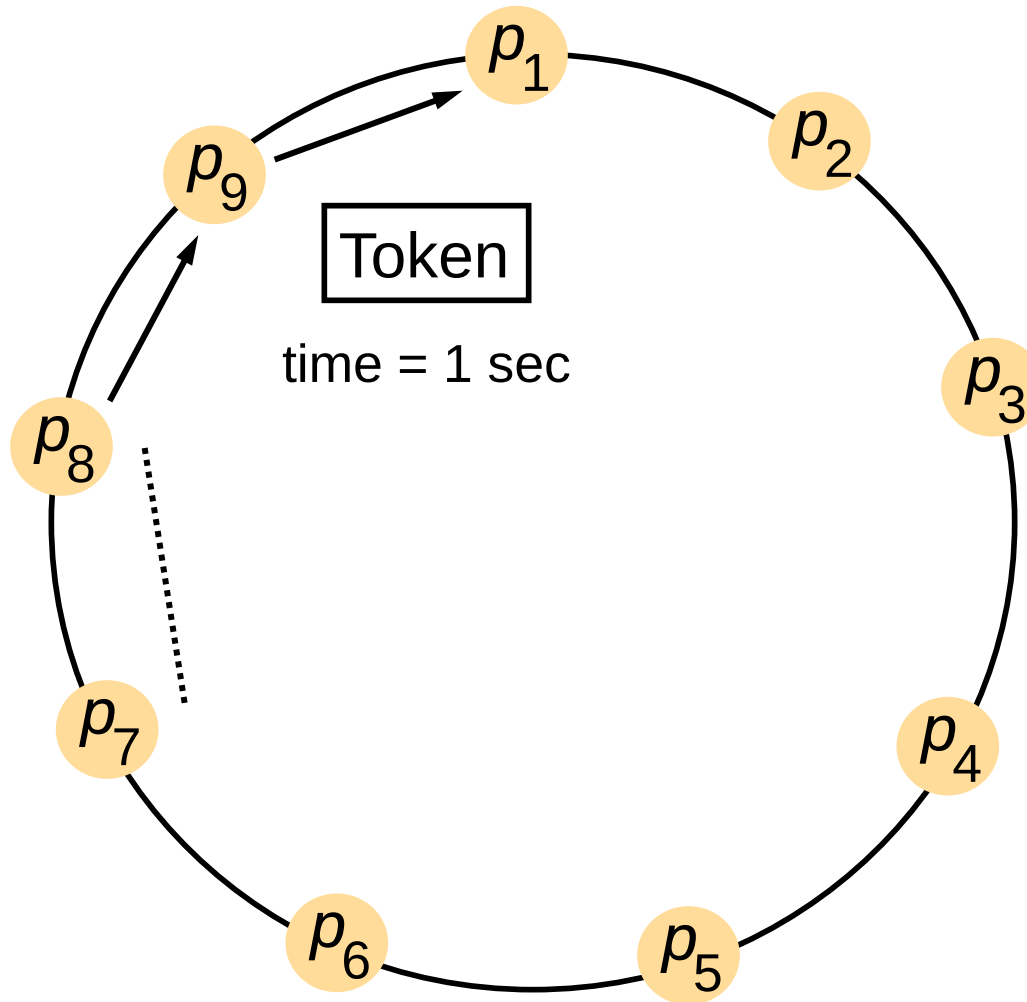


Ring-Based Algorithm

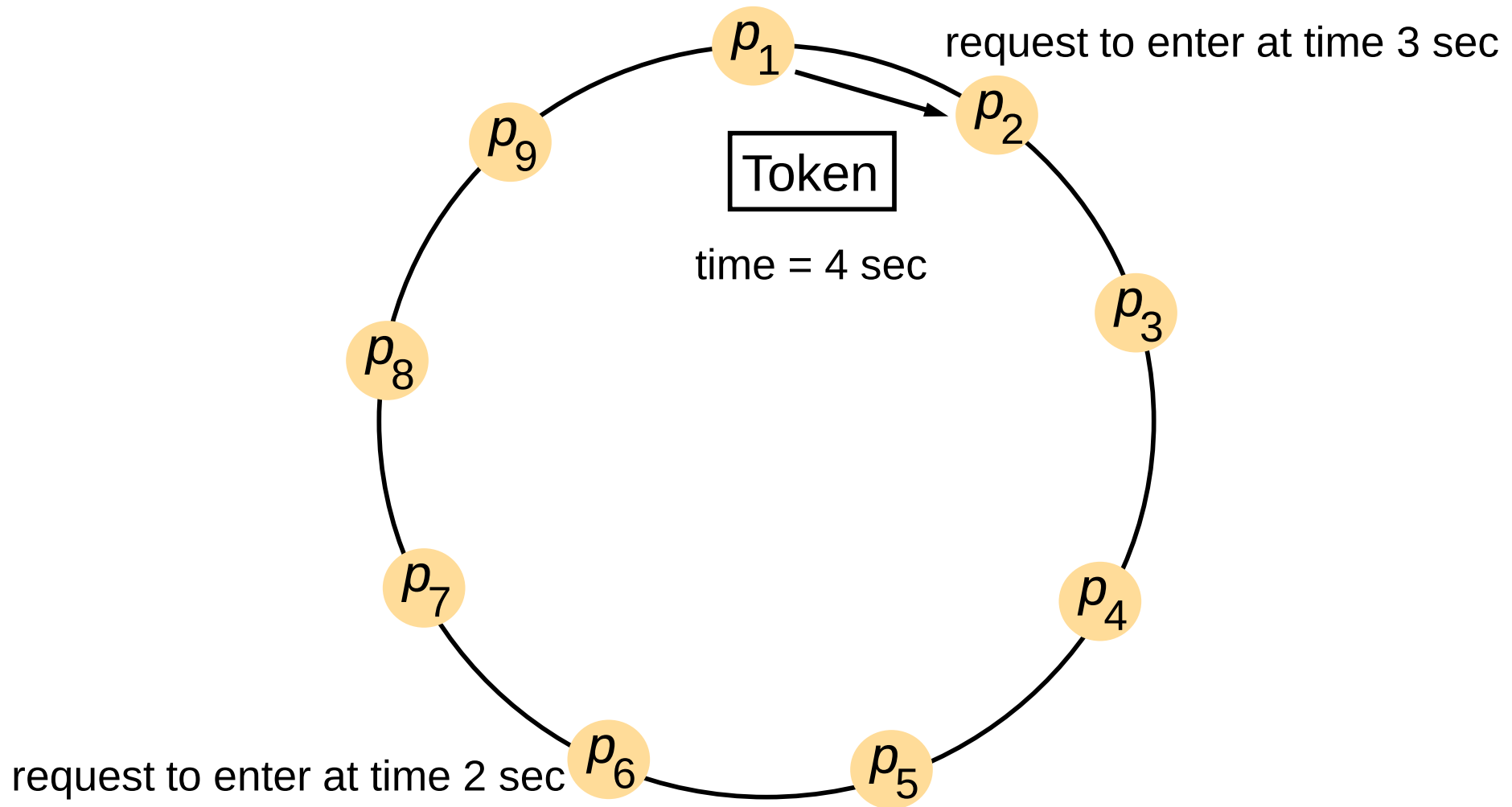
- A token is circulated in clockwise direction around the ring
 - On receiving the token, if the process does not require to enter the critical section, it immediately forwards the token to its neighbor
 - If the receiving process requires to enter the critical section, it retains the token and enters the critical section
 - To exit the critical section, the process sends the token on to its neighbor

Example of Ring-Based Algorithm

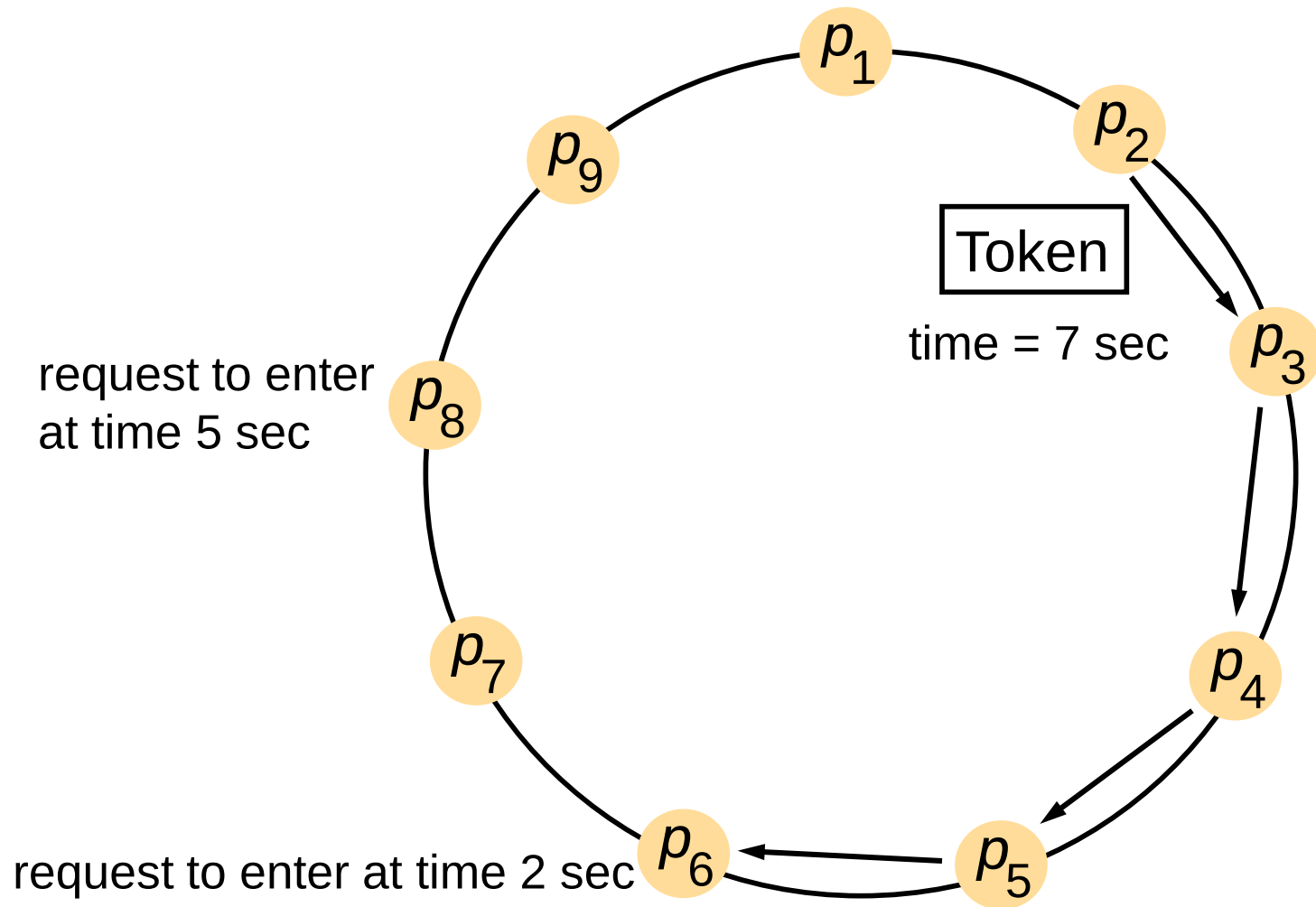
request to enter at time 1 sec



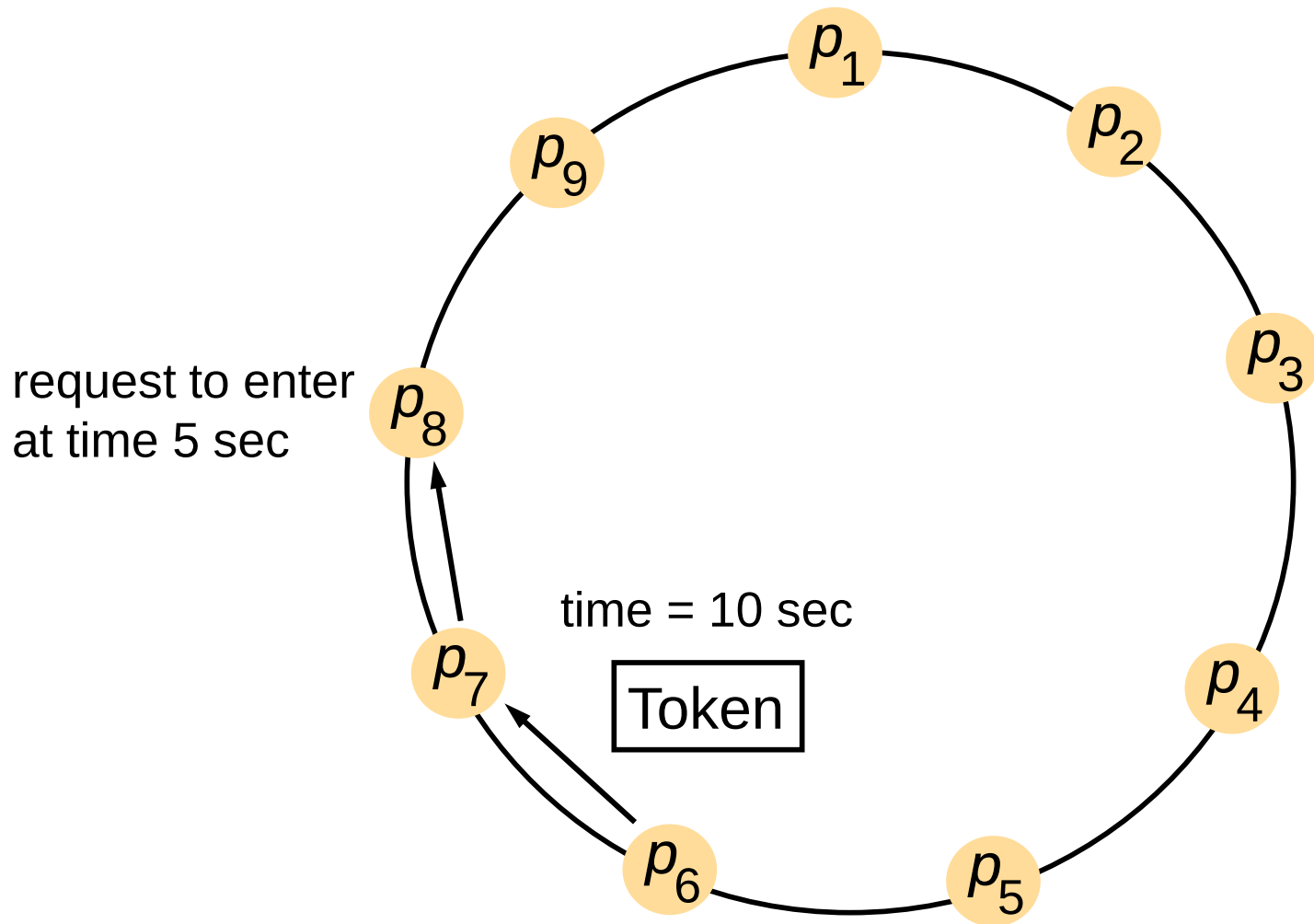
Example of Ring-Based Algorithm



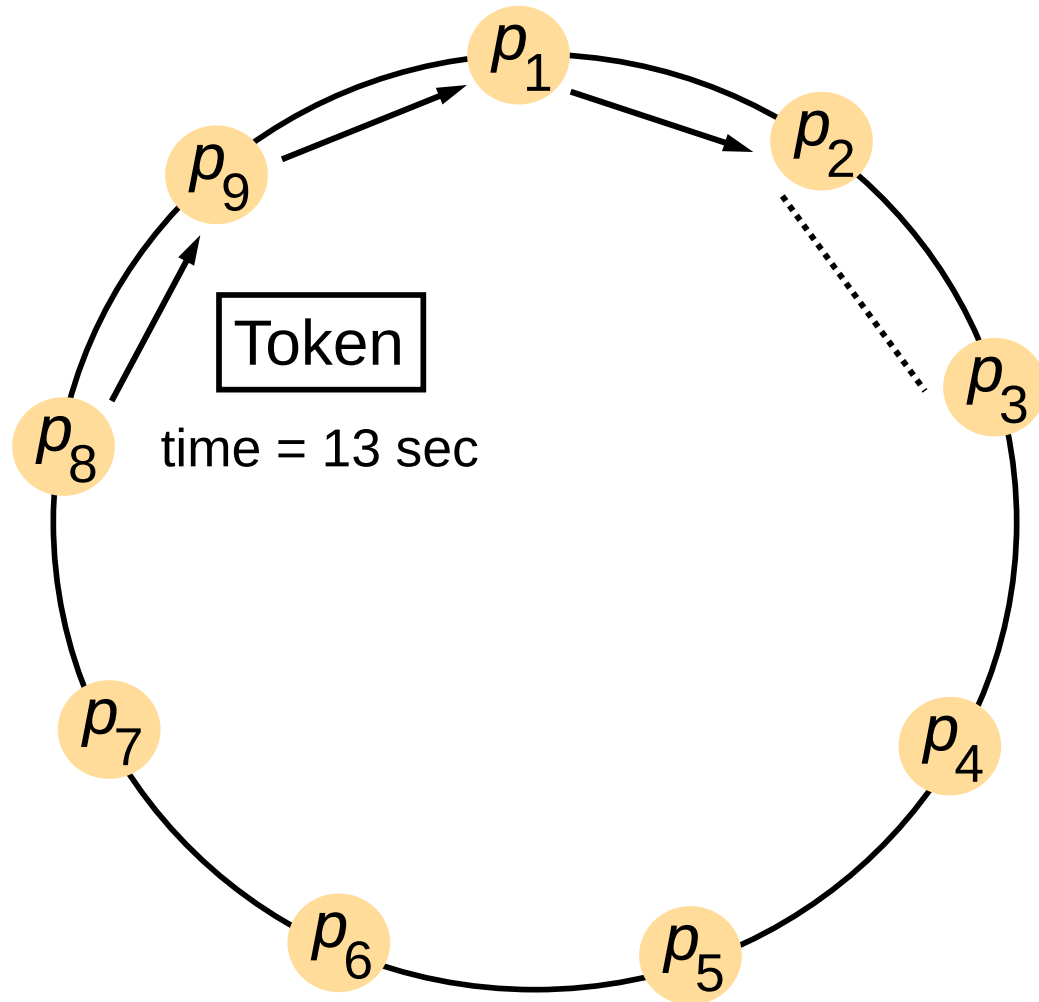
Example of Ring-Based Algorithm



Example of Ring-Based Algorithm



Example of Ring-Based Algorithm



Analysis of Ring-Based Algorithm

- Safety – satisfied
 - Only one token in the system → only one process can execute in the critical section at a time
- Liveness – satisfied
 - Token is circulated among all processes → it will find outstanding requests, so progress will always be made
 - No process can enter the critical section again without passing the token to the others first (hence granting others a chance) → no starvation
- Entry is not necessarily granted in the time order of requests
 - It depends on process locations in the ring

Analysis of Ring-Based Algorithm

- Bandwidth consumption
 - If every process constantly wants to enter a critical section, each token pass results in one entry and exit
 - On the other hand, processes send the token around the ring even when no process requires entry to the critical section → in this case, the number of messages per entry into a critical section is unbounded
- Client delay
 - Entering a critical section delays the process **between 0** (when it has just received the token) **and N message transmissions** (when it has just passed on the token)
- Synchronization delay
 - **Between 1 and N message transmissions**

Ricart and Agrawala Algorithm

- Require a total ordering of all events
 - For example, use Lamport's logical clocks and break ties with process identifiers
- Each process is in one of the following three states:
 - **RELEASED** – outside the critical section
 - **WANTED** – requiring entry to the critical section
 - **HELD** – executing in the critical section

Ricart and Agrawala Algorithm

- When a process requires entry to a critical section
 - Change to WANTED state
 - Send a request message to all other processes and wait for their replies
 - Can enter the critical section only when all other processes have replied to the request message
 - Change to HELD state when entering

Ricart and Agrawala Algorithm

- When a process receives a request message from another process
 - If the receiver is in RELEASED state, it replies immediately
 - If the receiver is in HELD state, it queues the request
 - If the receiver is in WANTED state, it compares the timestamp of the incoming request with its own request
 - If the incoming request has a smaller timestamp, reply immediately
 - Otherwise, queues the request

Ricart and Agrawala Algorithm

- When a process exits the critical section
 - Change to RELEASED state
 - Reply to all queued requests

Ricart and Agrawala Algorithm

On initialization

$state := \text{RELEASED};$

To enter the critical section

$state := \text{WANTED};$

send *request* to all the other processes;

$T :=$ timestamp of the request;

wait until (number of replies received = $(N - 1)$);

$state := \text{HELD};$

On receipt of a request $\langle T_j, p_j \rangle$ from another process p_j ($j \neq i$)

if ($state = \text{HELD}$ or ($state = \text{WANTED}$ and $(T, p_i) < (T_j, p_j)$))

then

 queue *request* from p_j without replying;

else

 reply immediately to p_j ;

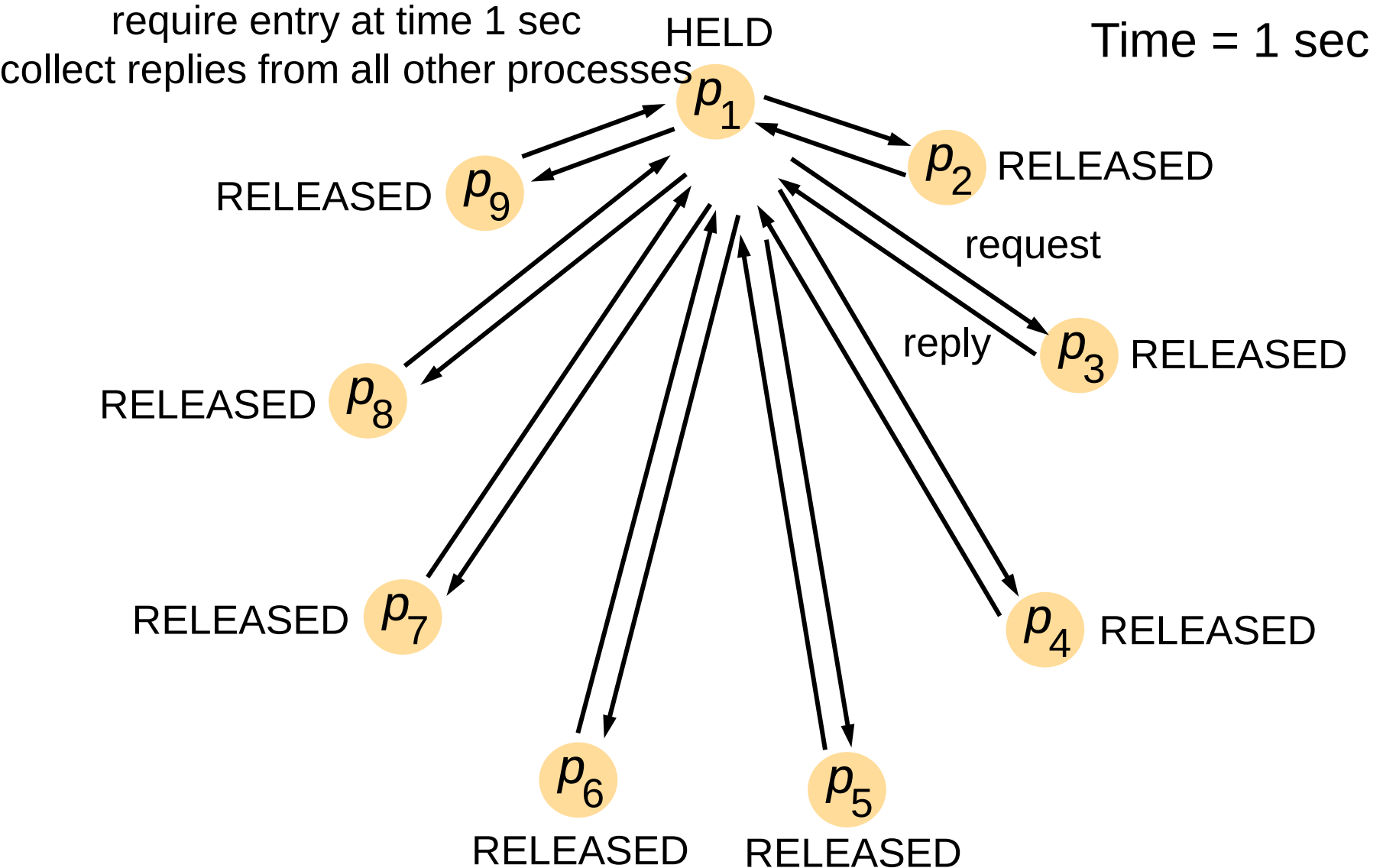
To exit the critical section

$state := \text{RELEASED};$

reply to all queued requests;

algorithm executed by a process p_i

Example of Ricart and Agrawala Algo

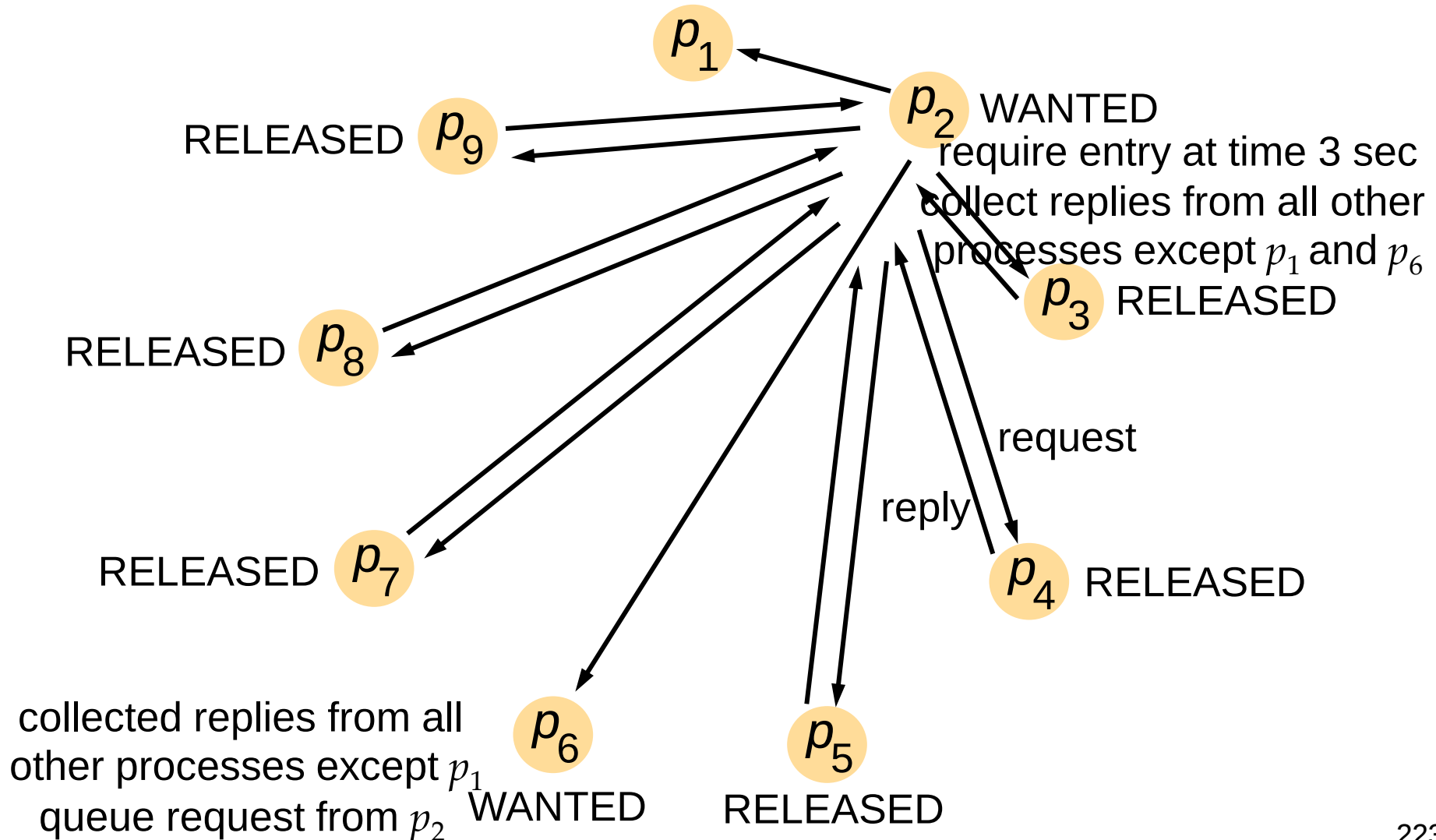




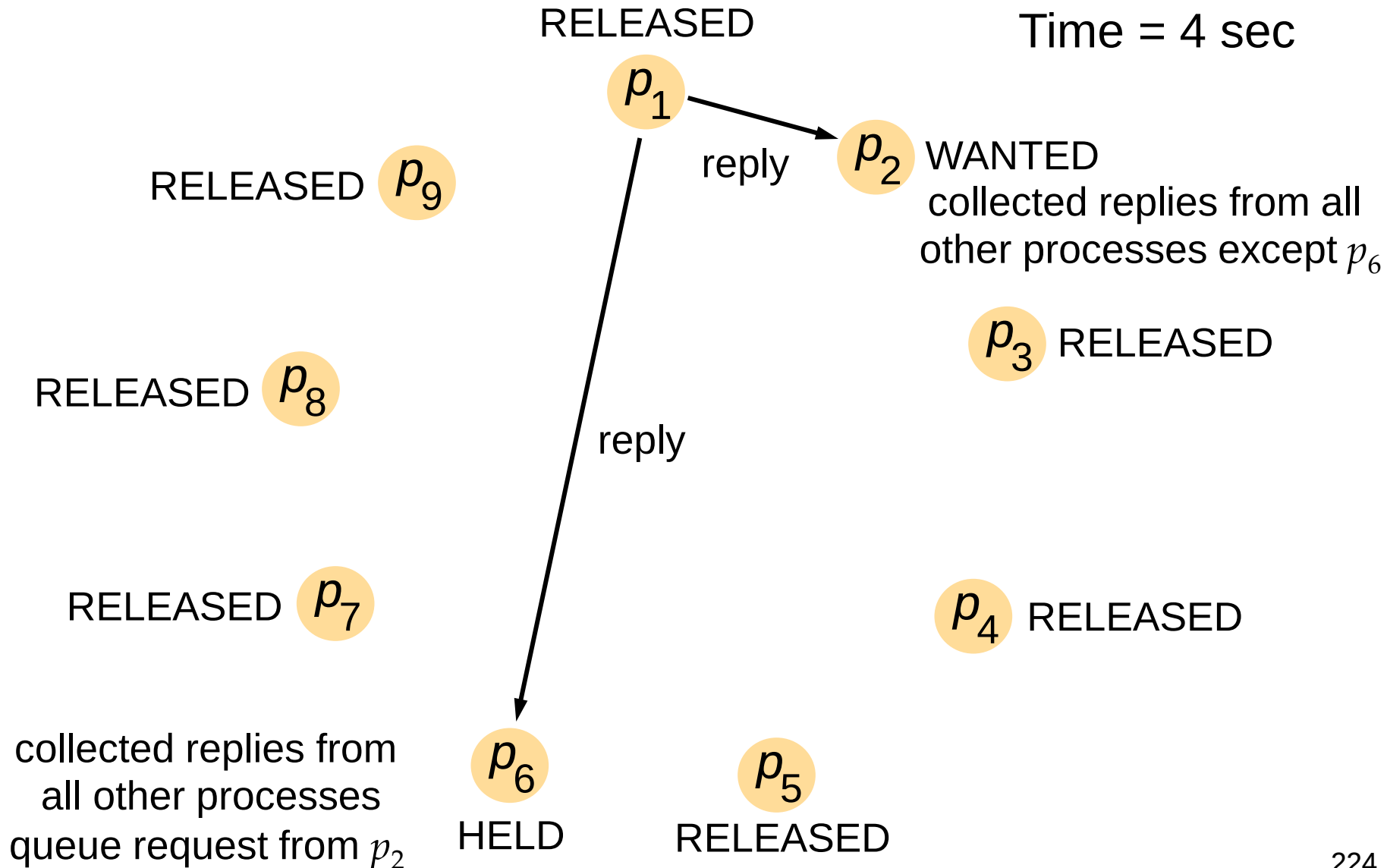
Example of Ricart and Agrawala Algo

queue requests from p_6 and p_2 HELD

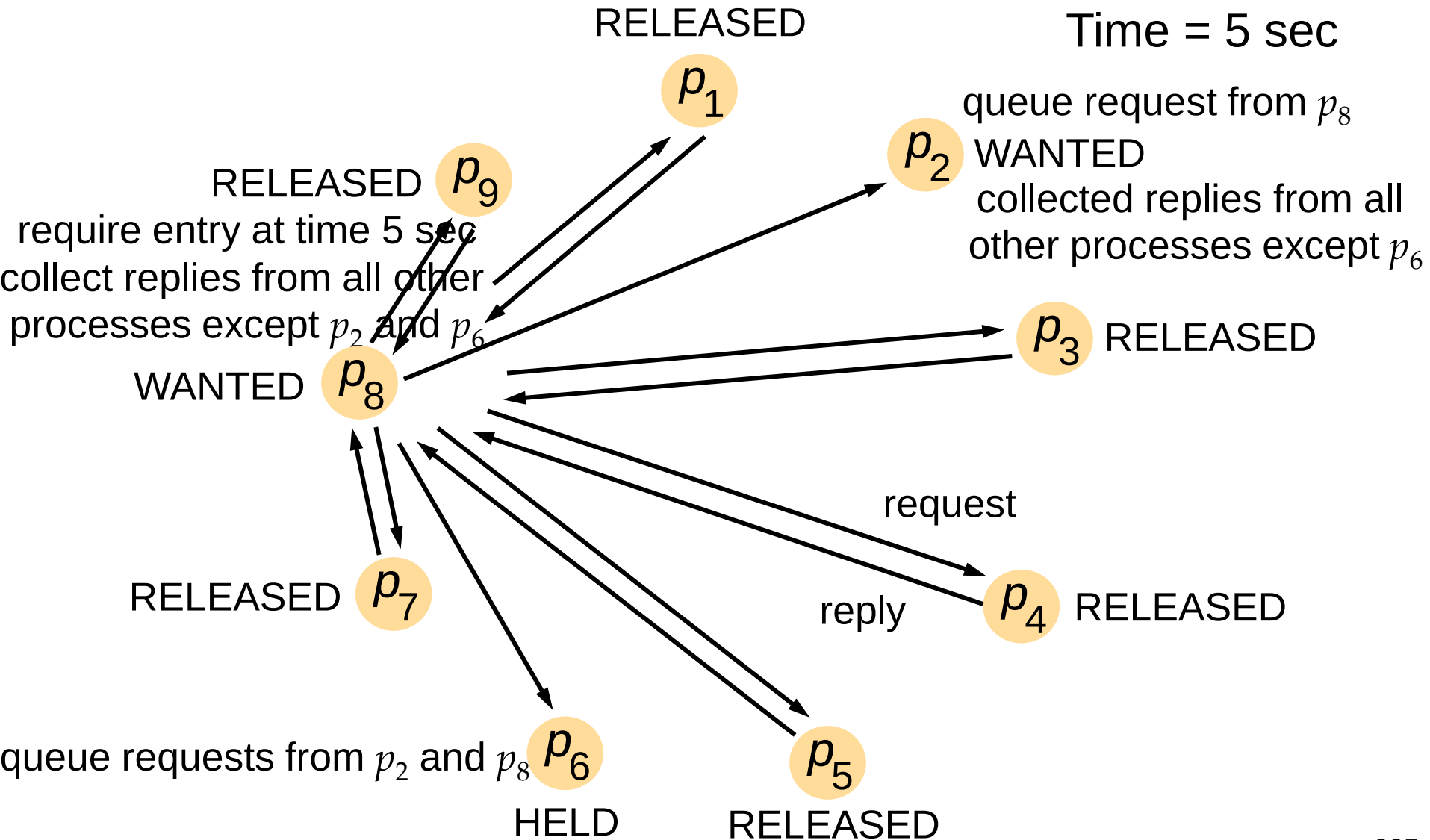
Time = 3 sec



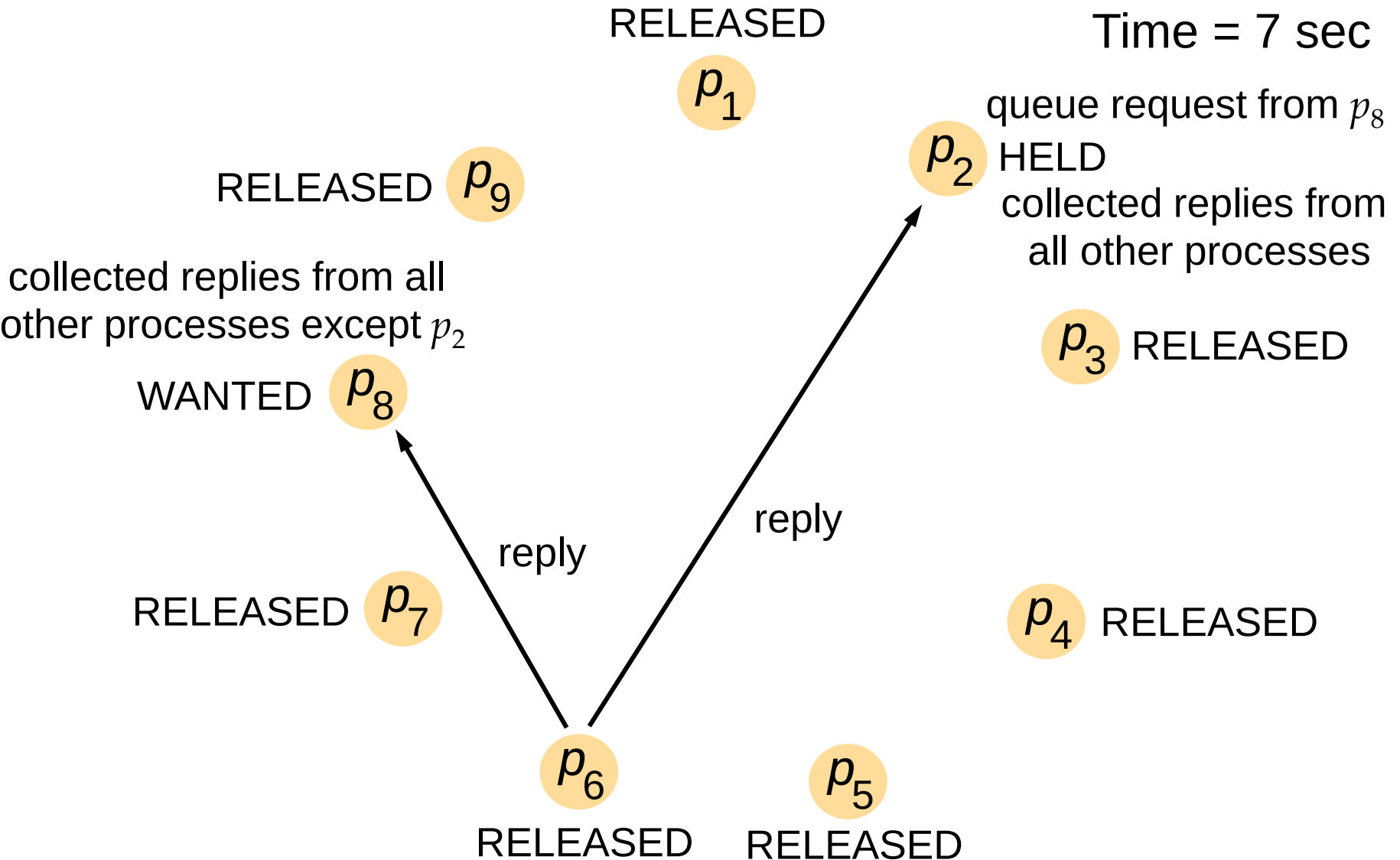
Example of Ricart and Agrawala Algo



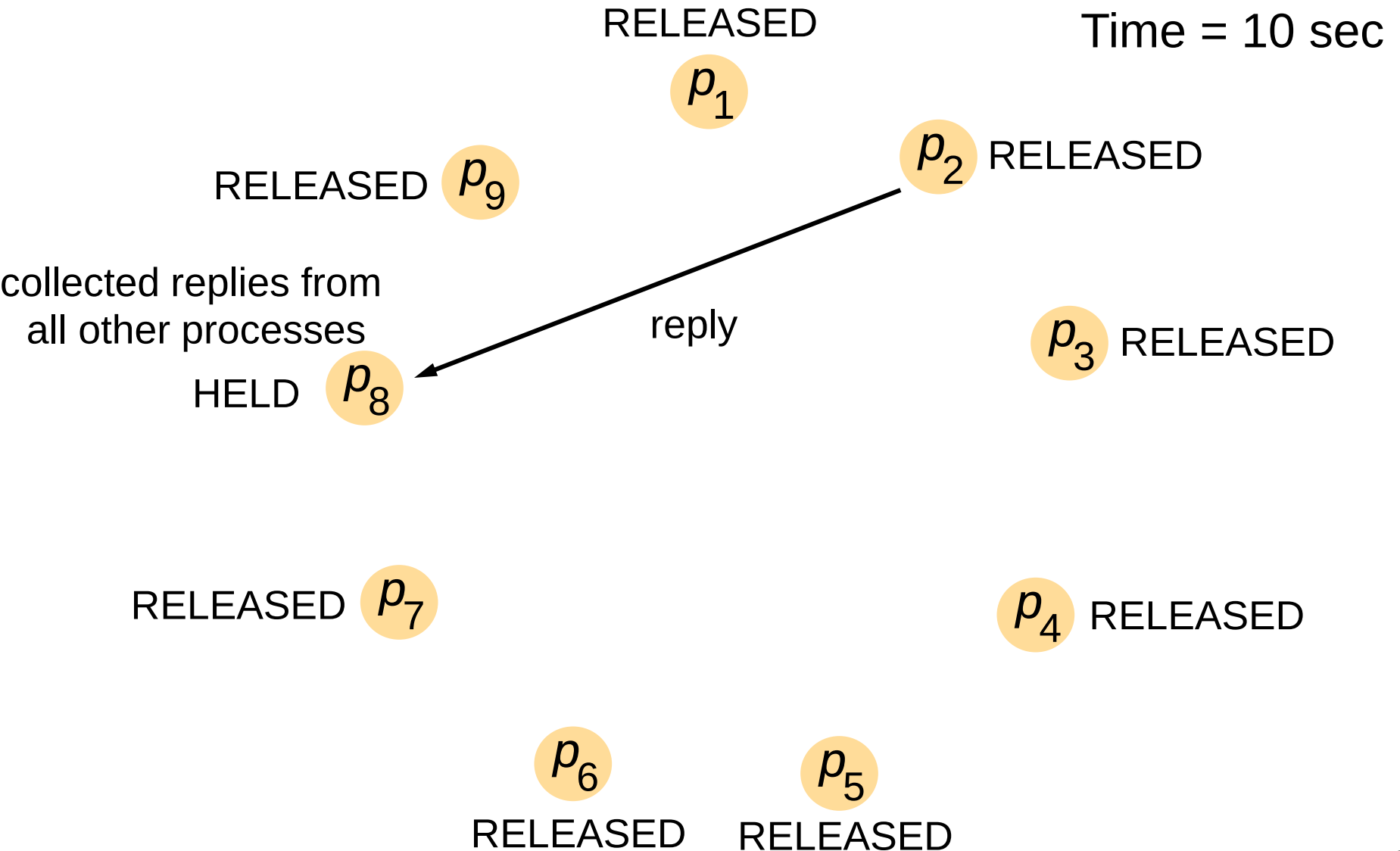
Example of Ricart and Agrawala Algo



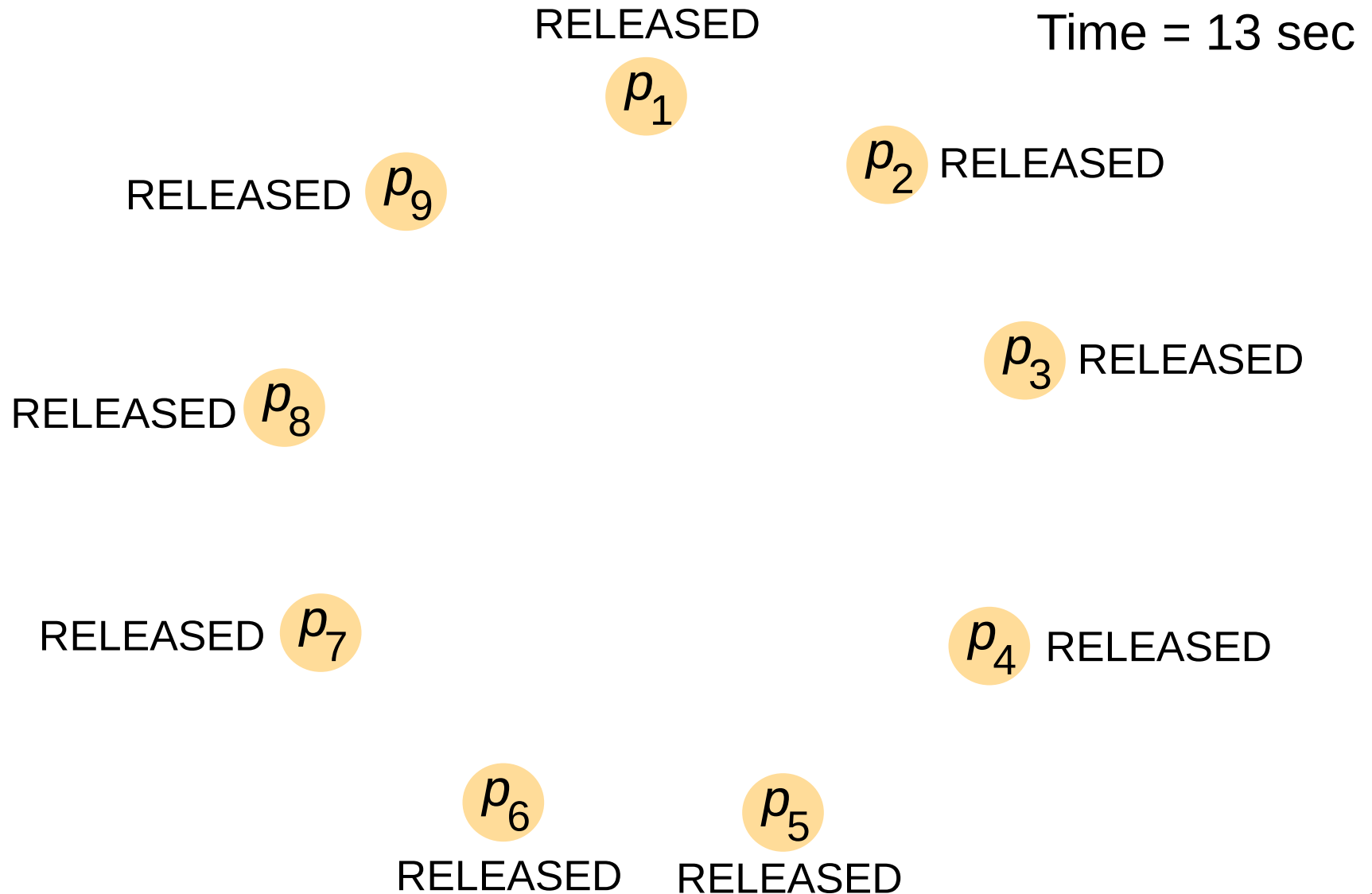
Example of Ricart and Agrawala Algo



Example of Ricart and Agrawala Algo



Example of Ricart and Agrawala Algo



Analysis of Ricart and Agrawala Algo

- Safety – satisfied
 - If it were possible for two processes to enter the critical section at the same time, both of them would have replied to the other, but since requests are totally ordered, this is impossible
- Liveness – satisfied
 - The outstanding request bearing the lowest timestamp would collect all replies and gain entry
- If Lamport's logical clocks are used, processes are granted entry to the critical section in causal ordering of their requests

Analysis of Ricart and Agrawala Algo

- Bandwidth consumption
 - Entering a critical section takes $2(N-1)$ messages ($N-1$ requests + $N-1$ replies)
- Client delay
 - Entering a critical section delays the process by a round-trip time (if all requests are sent simultaneously)
- Synchronization delay
 - 1 message transmission

Analysis of Fault Tolerance

- We have assumed that processes do not fail
- What if processes can fail by crashing only? – a good enough assumption for many systems
- Central server algorithm
 - Can tolerate the crash of a client process that neither holds nor has requested the token
- Ring-based algorithm, Ricart and Agrawala algorithm
 - Cannot tolerate the crash of any single process

Outline

- Distributed Mutual Exclusion
- Election
- Consensus Problem

Election

- Objective
 - To choose a **unique** process (called **coordinator**) to play a particular role
 - e.g., a time server among a set of computers
 - e.g., a server in the central server algorithm for mutual exclusion if it is one of the N processes $p_1, p_2, \dots, p_N$ to be coordinated
 - In general, a coordinator may offer any service such as synchronization, breaking deadlocks etc.
- If the current coordinator crashes or wishes to retire, an election is required to choose a new coordinator on which all processes agree

Election

- A process initiates the election if it starts a particular run of the election algorithm
 - An individual process does not initiate more than one election at a time, but **different processes may call elections concurrently** → we need to ensure that a unique coordinator is elected in such cases
 - **More processes may fail during election** → we need to ensure that a non-crashed process is eventually elected in such cases

Election

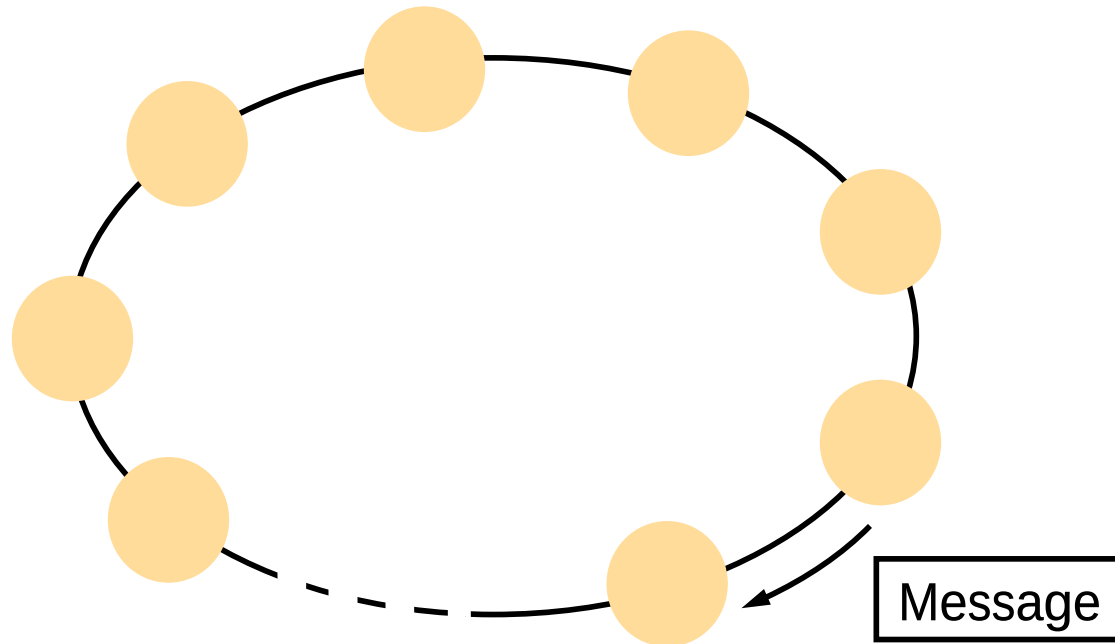
- Assume each process has a unique identifier
- Without loss of generality, we require the coordinator be chosen as the process with largest identifier
 - Assume a total order of process identifiers
 - The identifier may be any useful value, e.g., to elect a process with the lowest computational load, we can take $1/load$ as the process identifier (or use $\langle 1/load, i \rangle$ to break ties where i is the process index)

Performance Metrics

- Bandwidth consumption
 - Total number of messages sent
- Turnaround time
 - The number of serialized message transmission times between the initiation and termination of an election

Ring-Based Algorithm

- Arrange processes in a logical ring
 - Each process knows the next process around the ring
 - All messages are sent clockwise around the ring



Ring-Based Algorithm

- Assume asynchronous system and no failure occurs during election
- A process is called a **participant** if it is currently engaged in some election(s), otherwise the process is called a **non-participant**
- Initially, every process is marked as a non-participant in an election
- Any process can initiate an election, it proceeds by marking itself as a participant, placing its identifier in **an election message** and sending it to its clockwise neighbor

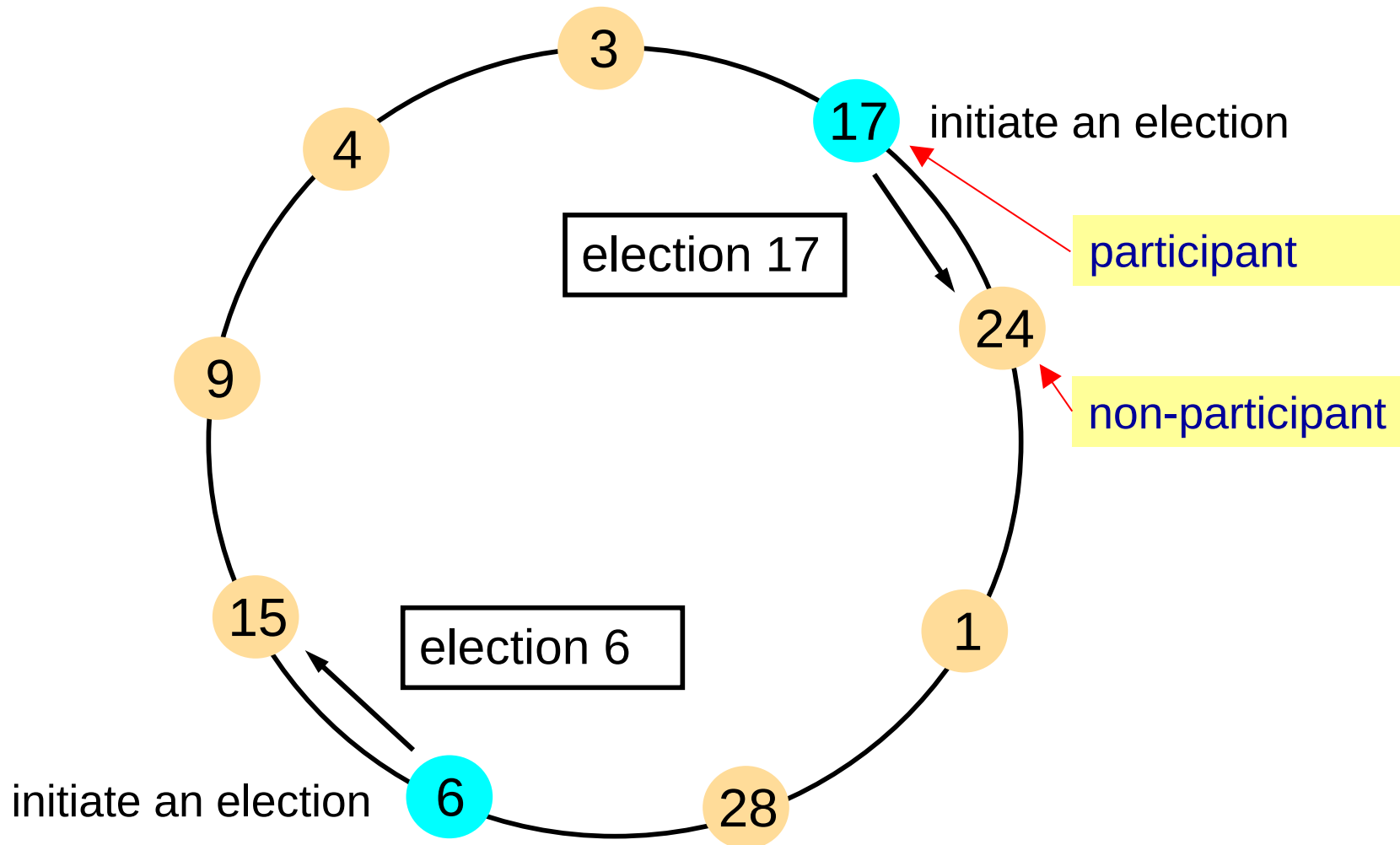
Ring-Based Algorithm

- When a process receives **an election message**, it compares the identifier in the message with its own identifier
 - If the arrived identifier is greater, it forwards the message to its neighbor
 - If the arrived identifier is smaller and the receiver is not a participant yet, it substitutes its own identifier in the message and forwards the message
 - If the arrived identifier is smaller and the receiver is already a participant, it does not forward the message
 - On forwarding an election message in any case, the process marks itself as a participant

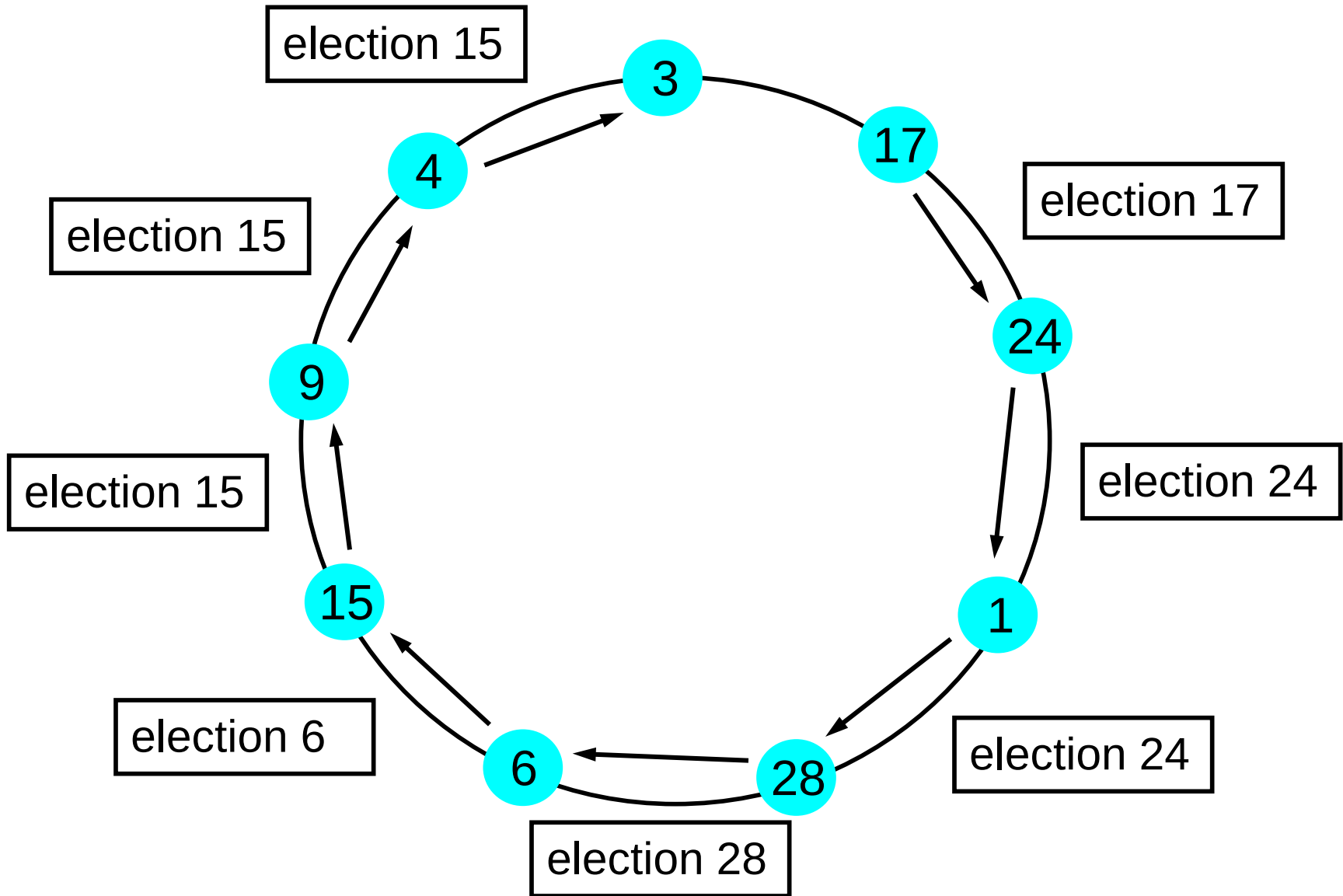
Ring-Based Algorithm

- (cont'd)
 - If the arrived identifier is that of the receiver itself, this process's identifier must be the greatest and it becomes the **coordinator**
- The coordinator marks itself as a non-participant and sends **an elected message** to its neighbor, announcing its election and enclosing its identity
 - When a process p_i receives an elected message, it marks itself as a non-participant and records the coordinator identity found in the message
 - p_i forwards the elected message to its neighbor unless it is the coordinator

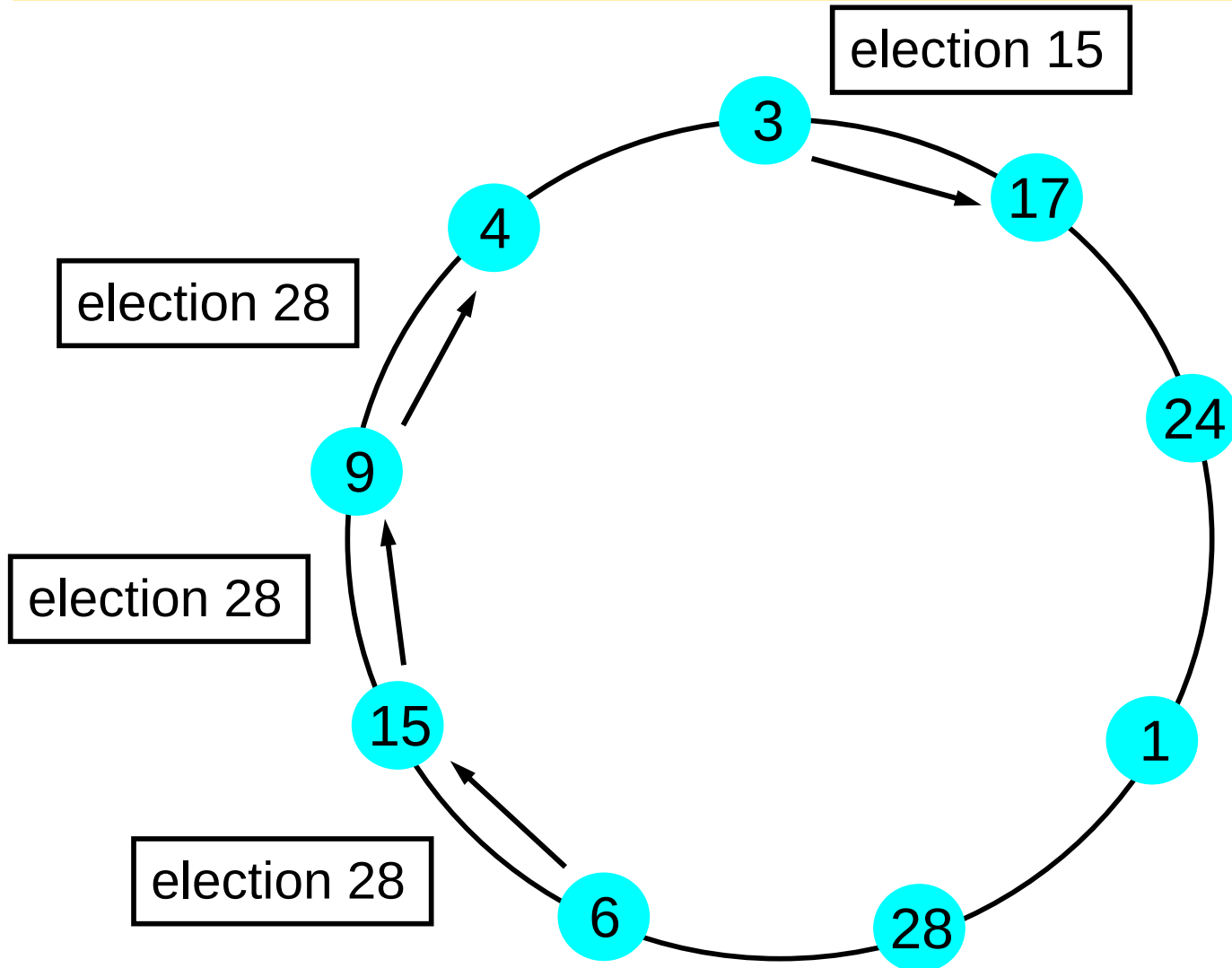
Example of Ring-Based Algorithm



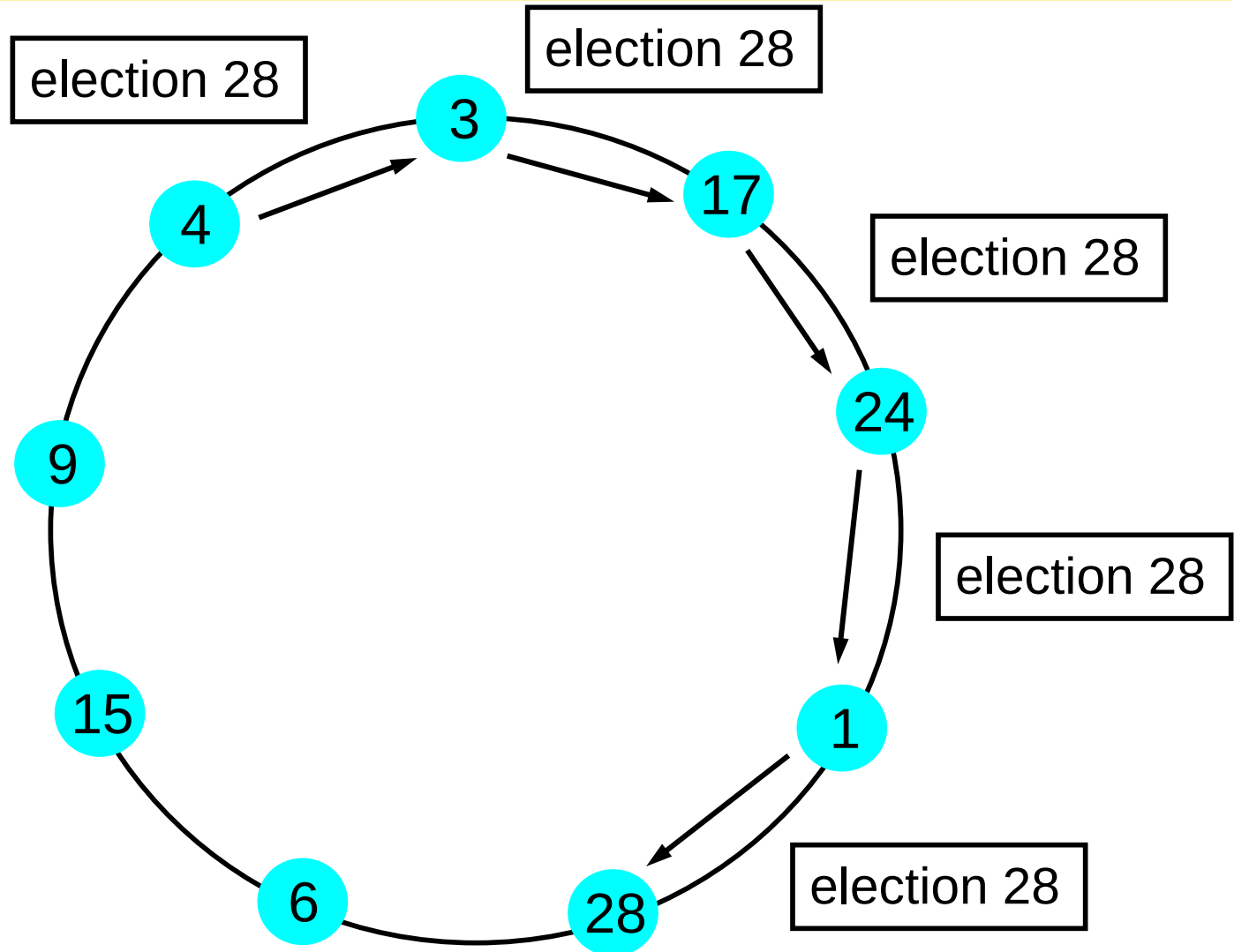
Example of Ring-Based Algorithm



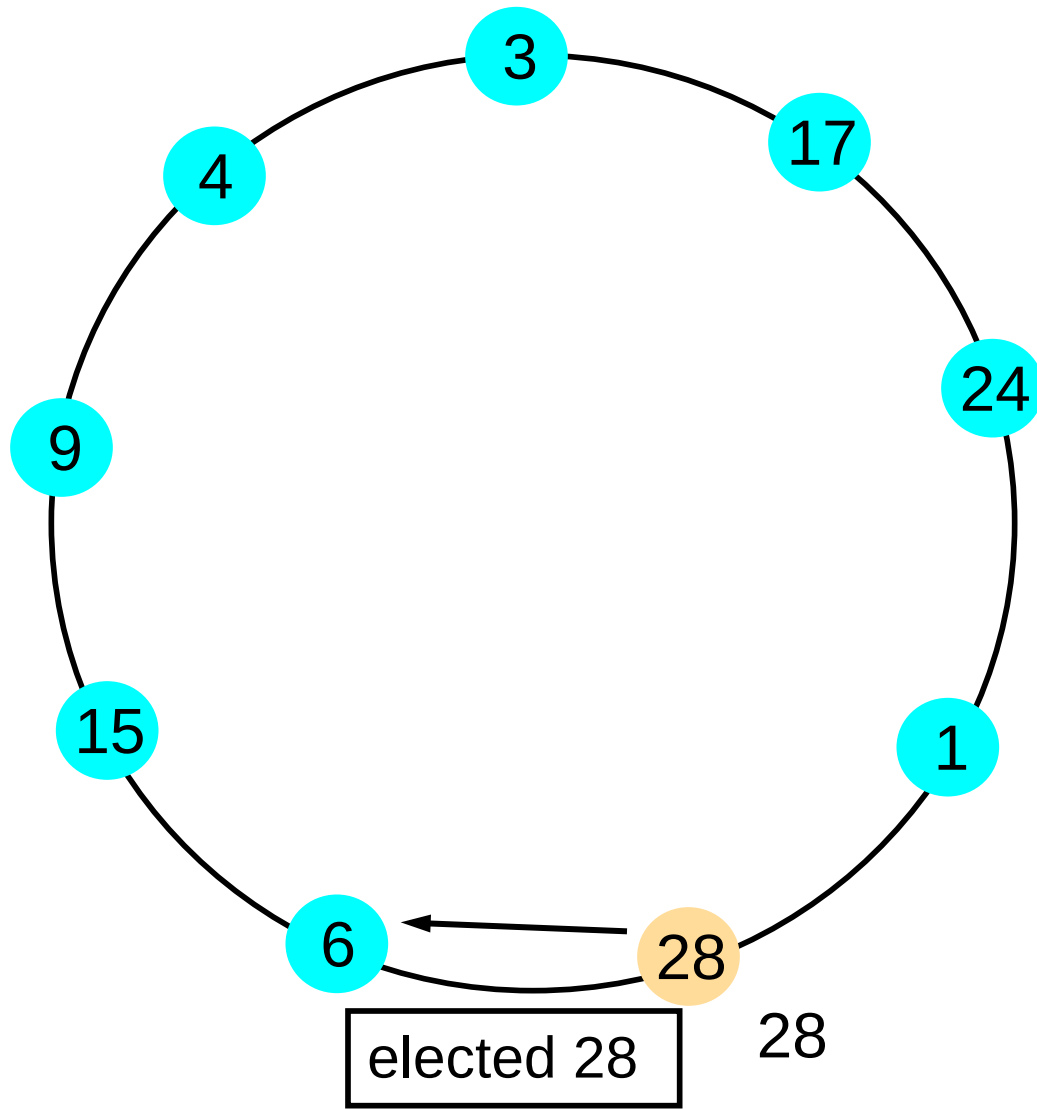
Example of Ring-Based Algorithm



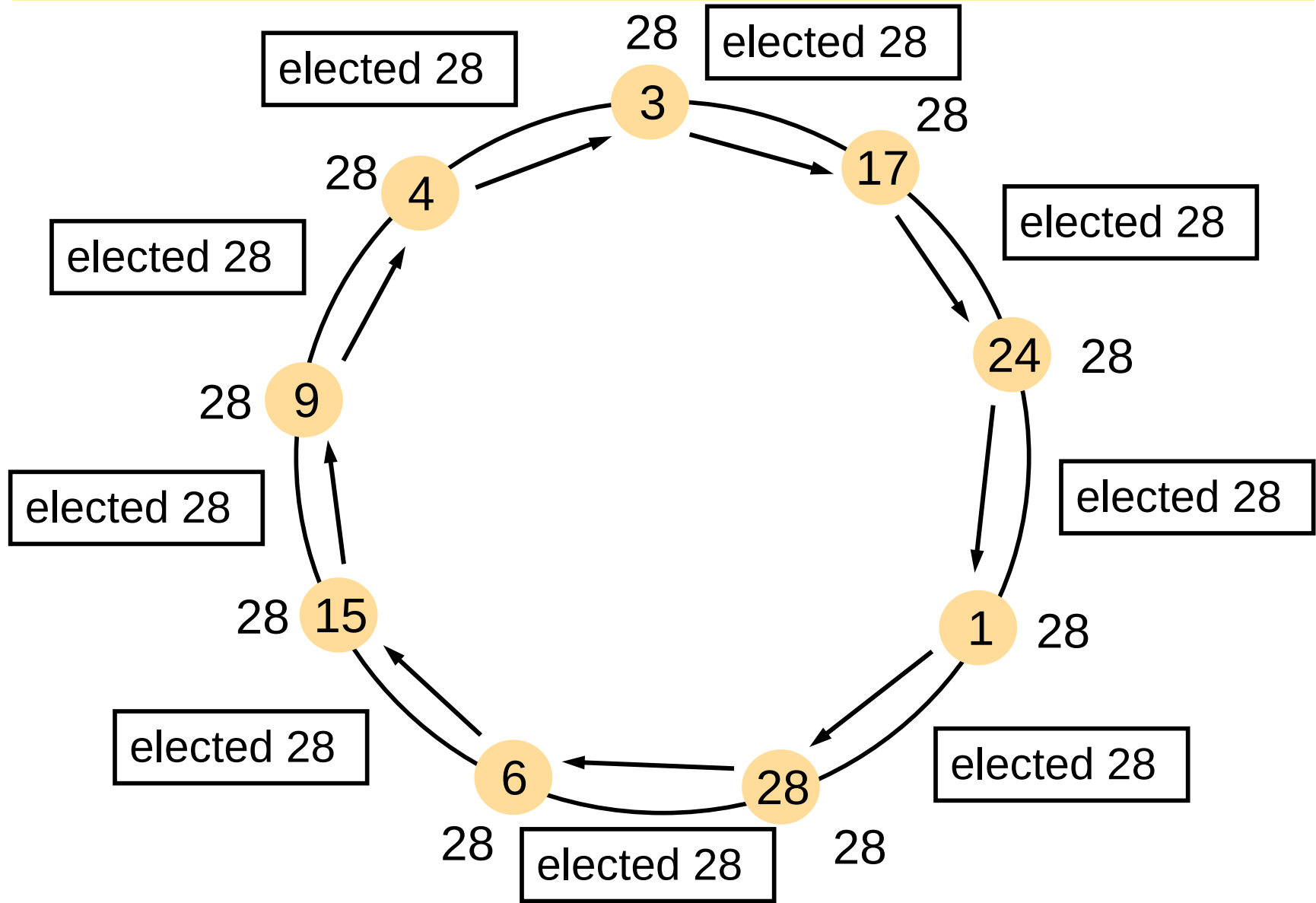
Example of Ring-Based Algorithm



Example of Ring-Based Algorithm



Example of Ring-Based Algorithm



Analysis of Ring-Based Algorithm

■ Correctness

- All identifiers are compared since a process must receive its own identifier back before sending an elected message
- For any two processes, the one with the larger identifier will not pass on the other's identifier
→ it is impossible that both processes receive their own identifiers back
- So, only one process would receive its own identifier back and this process's identifier must be the largest

Analysis of Ring-Based Algorithm

■ Bandwidth consumption

- Assume only a single process initiates an election
- Worst case: when its anti-clockwise neighbor has the largest identifier → it takes $N-1$ election messages to reach this neighbor
- Then, N more election messages to complete a circuit before announcing the coordinator
- Finally, announcing the coordinator takes further N elected messages
- So, a total of $3N-1$ messages

■ Turnaround time

- Worst case: $3N-1$ message transmissions since these messages are sent sequentially

Bully Algorithm

- Allow processes to crash during an election
- Assume synchronous system and use timeouts to detect process crash
 - Let T_{trans} be a maximum message transmission delay, and T_{process} be a maximum time for processing a message, then an upper bound on the total elapsed time from sending a message to another process to receiving a response is $T = 2 T_{\text{trans}} + T_{\text{process}}$
 - If no response arrives within time T , the intended recipient must have crashed

Bully Algorithm

- Assume each process knows which processes have larger identifiers and it can communicate with all such processes
- Three types of messages in Bully algorithm
 - **Election message:** to announce an election
 - **Answer message:** to respond to an election message
 - **Coordinator message:** to announce the identity of the coordinator
- When any process detects that the coordinator crashes, it begins an election
 - Several processes may discover this concurrently

Bully Algorithm

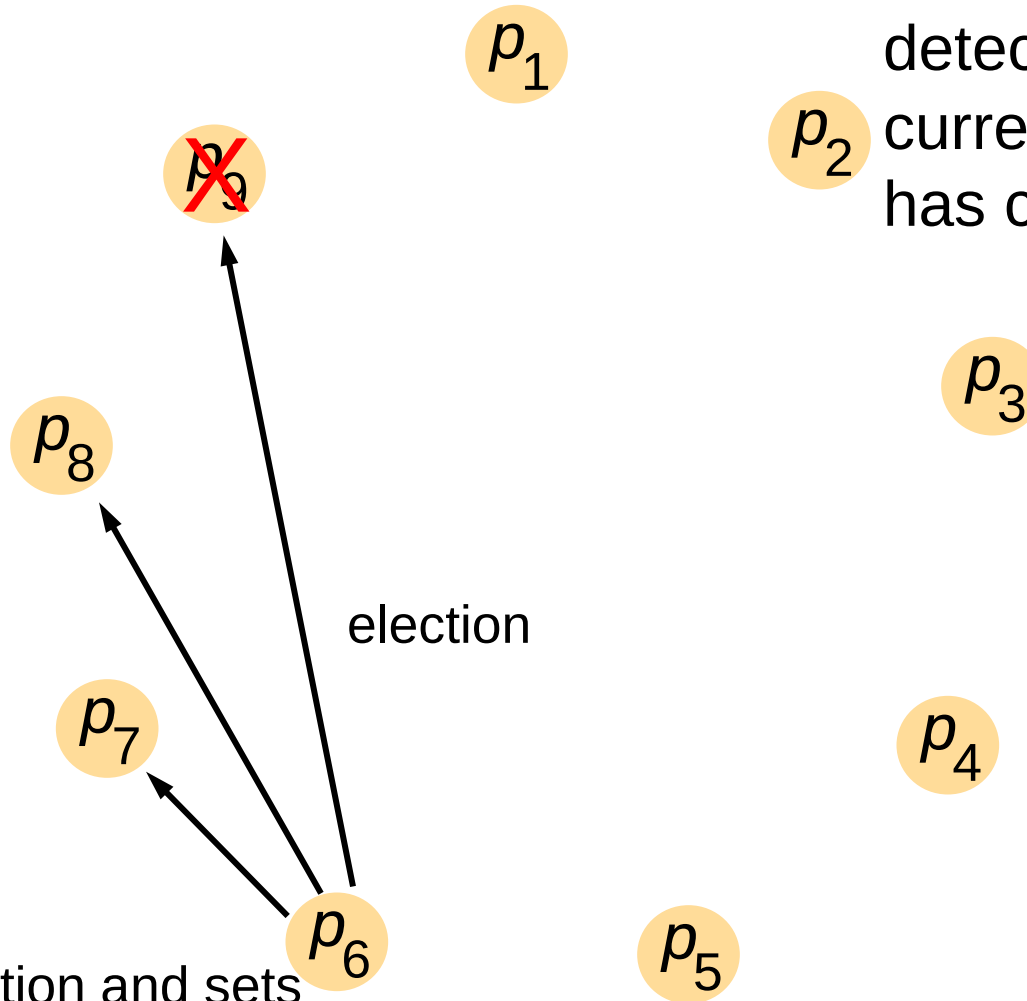
- When a process P begins an election
 - P sends **an election message** to all processes with larger identifiers and awaits **answer messages** in response
 - If no process responds within time T , P wins the election and becomes the coordinator (to announce this, P sends **a coordinator message** to all processes with smaller identifiers)
 - If some process responds with answer message, P waits a further period for a coordinator message to arrive from the new coordinator (if none arrives, P begins another election)
 - When P is waiting for answer messages, P is said to hold an election

Bully Algorithm

- When a process receives **an election message** (from another process with smaller identifier)
 - The receiving process sends back **an answer message** to indicate that it is alive
 - The receiving process begins an election (as described in the previous slide) if it is not holding an election
- If a crashed process comes back up
 - If it is the process with the largest identifier, it decides that it is the coordinator and announces this to all other processes by sending **a coordinator message**
 - Otherwise, it begins an election

Example of Bully Algorithm

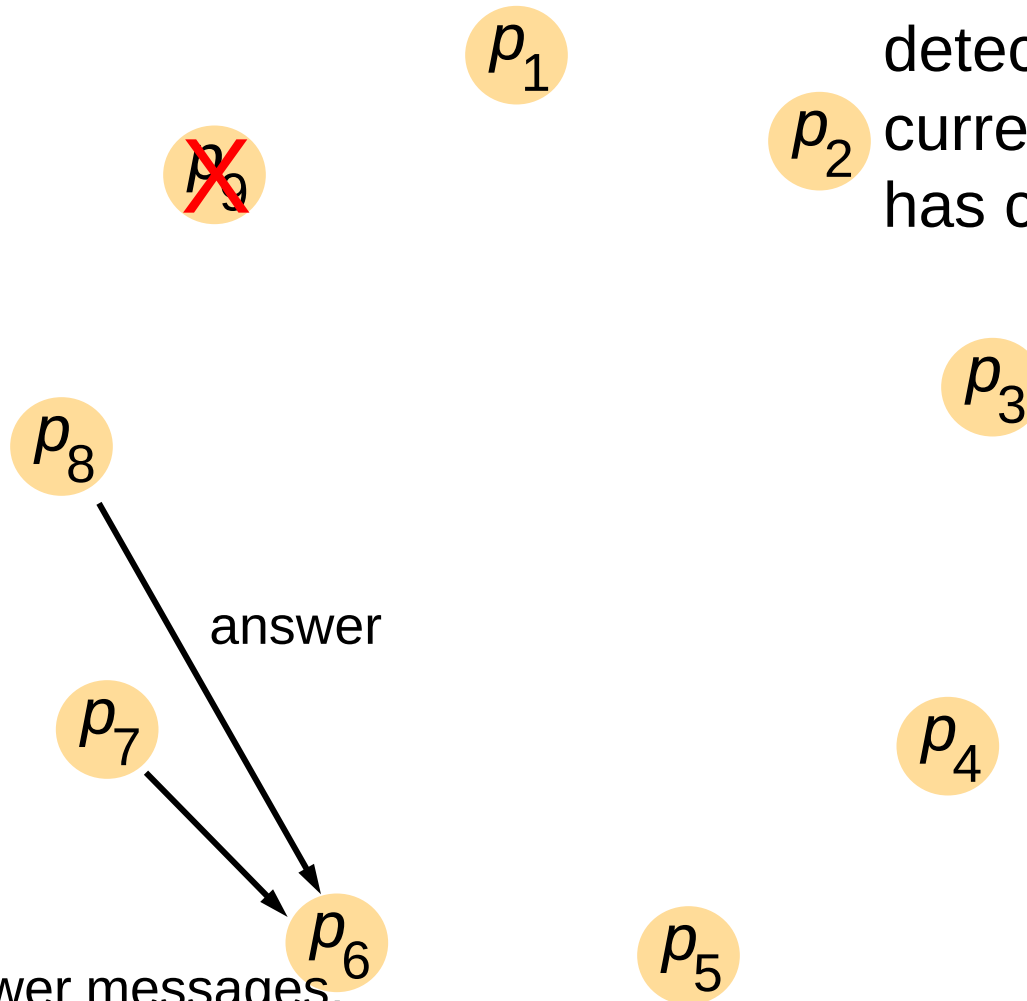
Example 1: if p_6 detects that p_9 (the current coordinator) has crashed



p_6 begins an election and sets a timeout T for answer messages

Example of Bully Algorithm

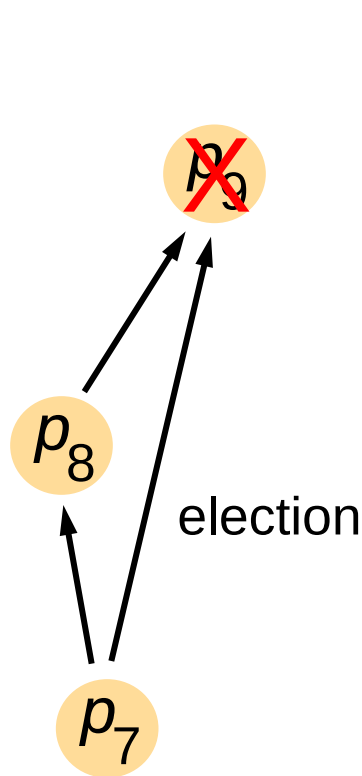
Example 1: if p_6 detects that p_9 (the current coordinator) has crashed



on receiving answer messages,
 p_6 sets a timeout for the coordinator message

Example of Bully Algorithm

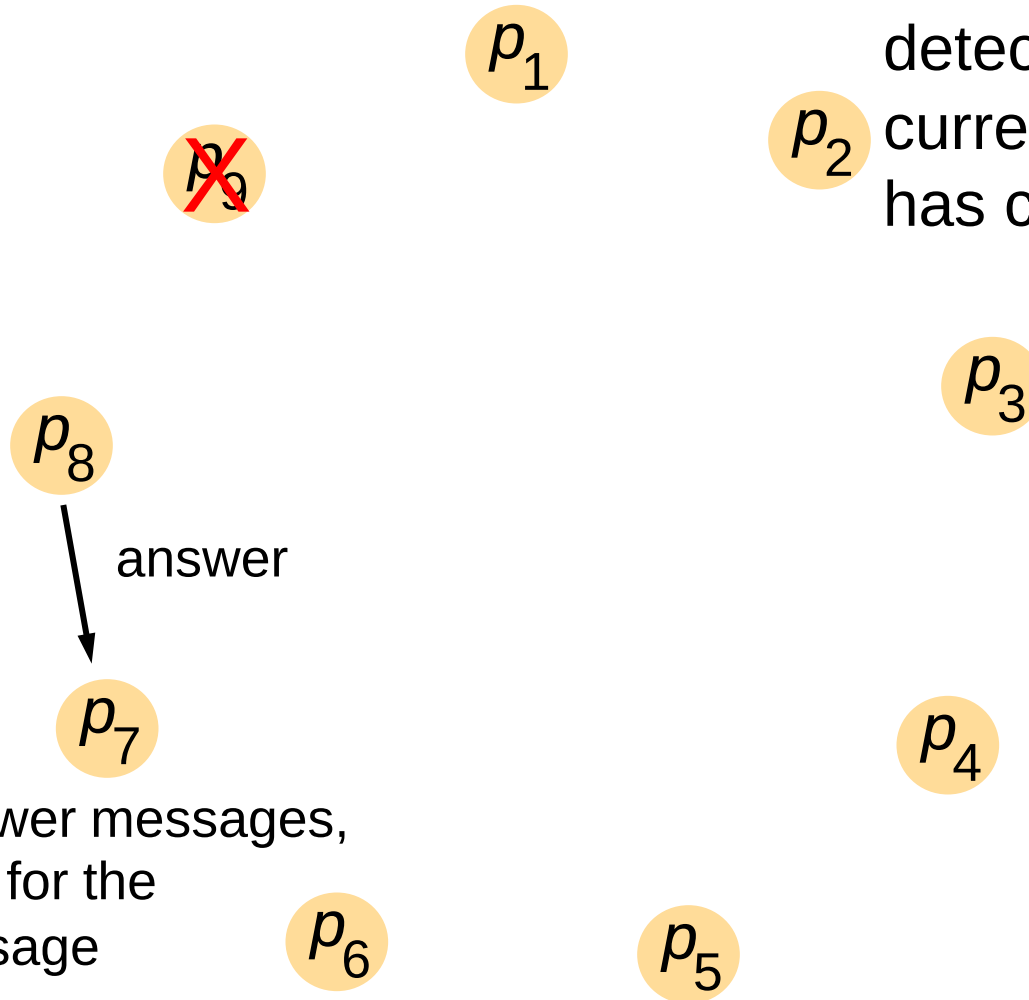
Example 1: if p_6 detects that p_9 (the current coordinator) has crashed



p_7 and p_8 begin elections since they are not holding elections; p_7 and p_8 also set timeout T for answer messages

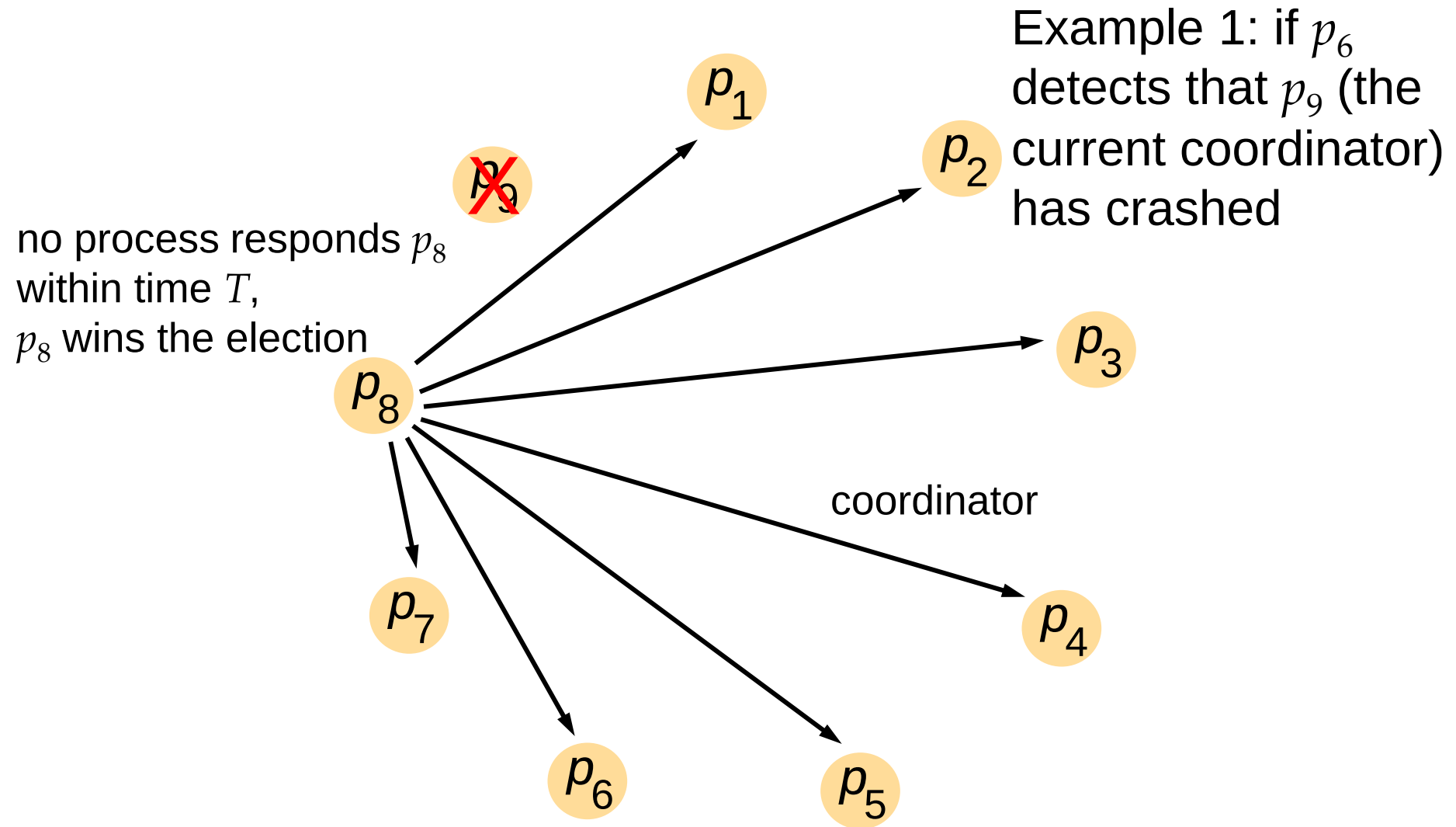
Example of Bully Algorithm

Example 1: if p_6 detects that p_9 (the current coordinator) has crashed



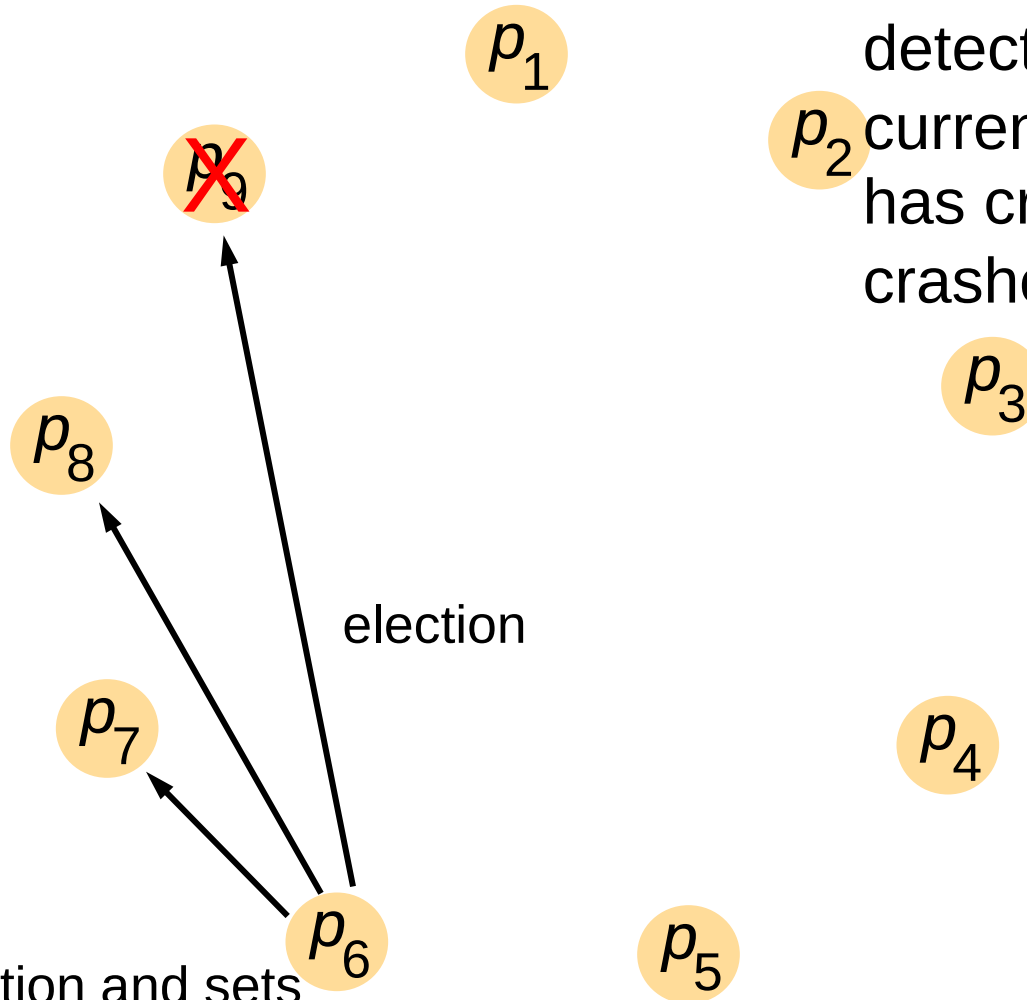
on receiving answer messages,
 p_7 sets a timeout for the
coordinator message

Example of Bully Algorithm



Example of Bully Algorithm

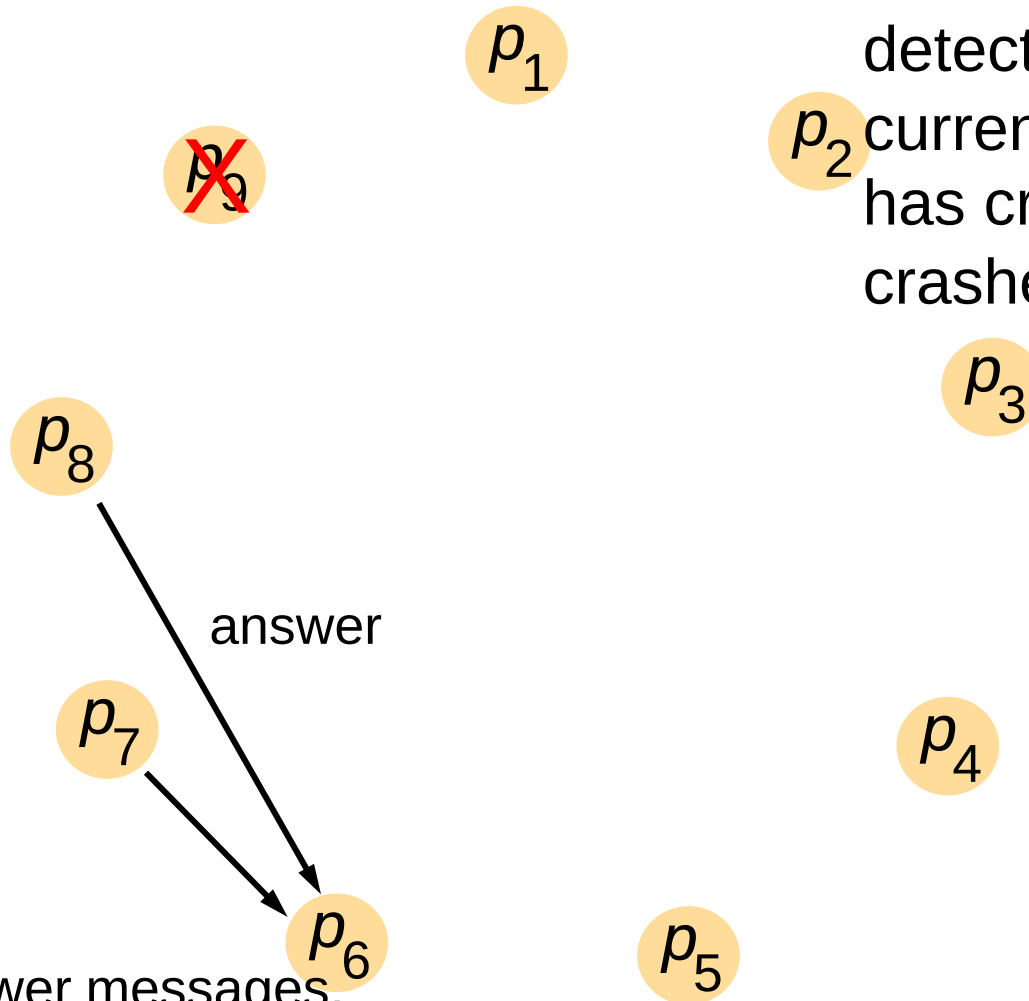
Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



p_6 begins an election and sets a timeout T for answer messages

Example of Bully Algorithm

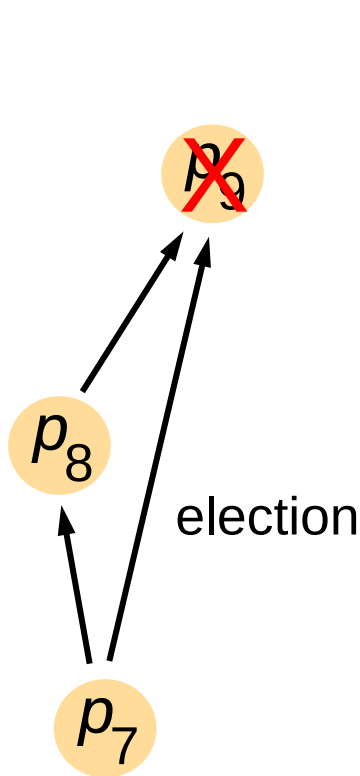
Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



on receiving answer messages,
 p_6 sets a timeout for the coordinator message

Example of Bully Algorithm

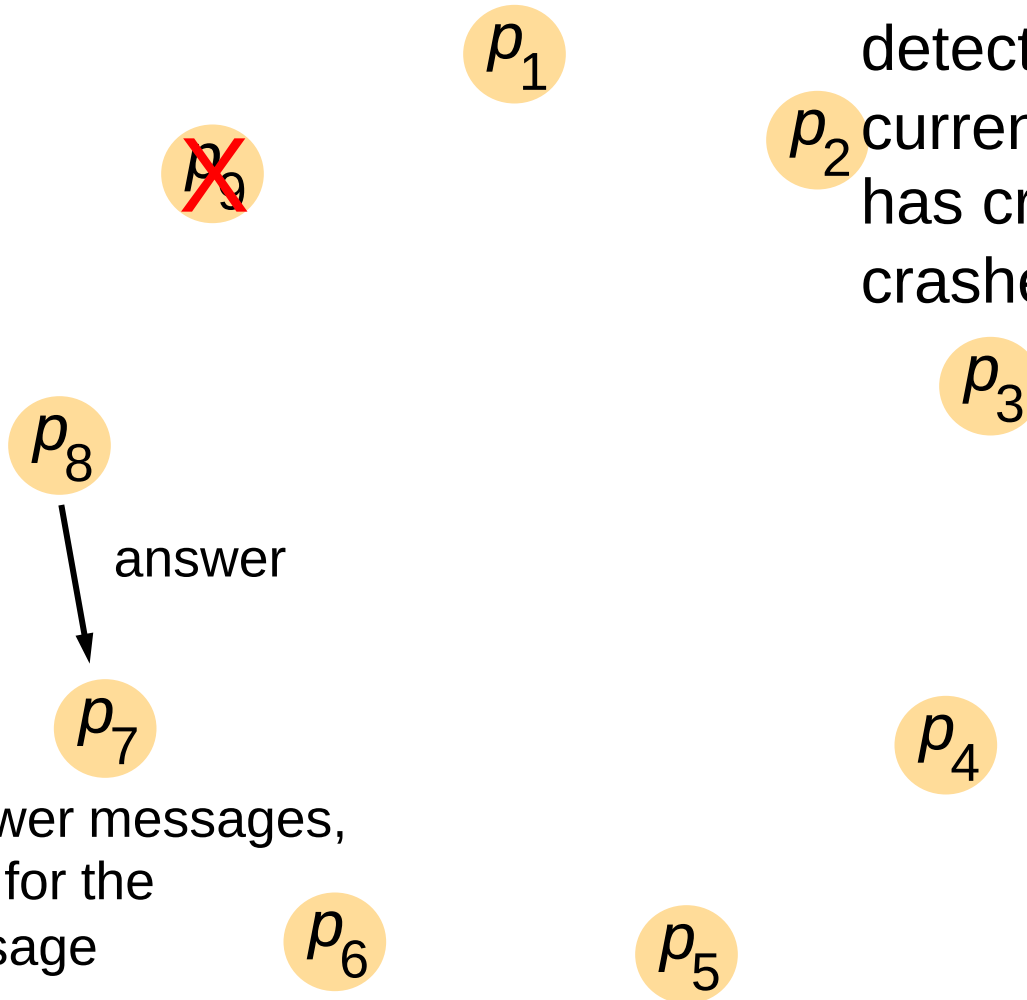
Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



p_7 and p_8 begin elections since they are not holding elections; p_7 and p_8 also set timeout T for answer messages

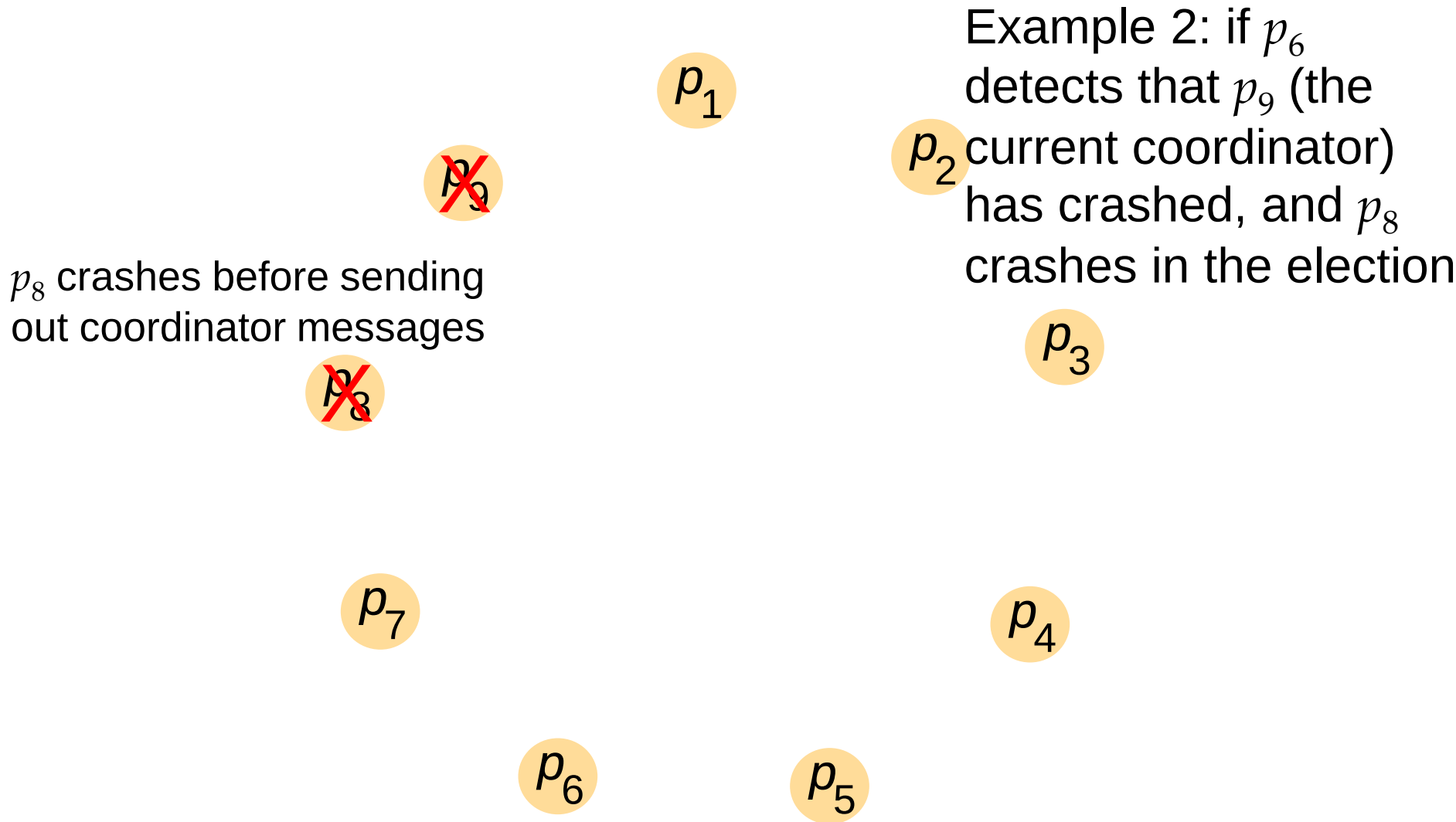
Example of Bully Algorithm

Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



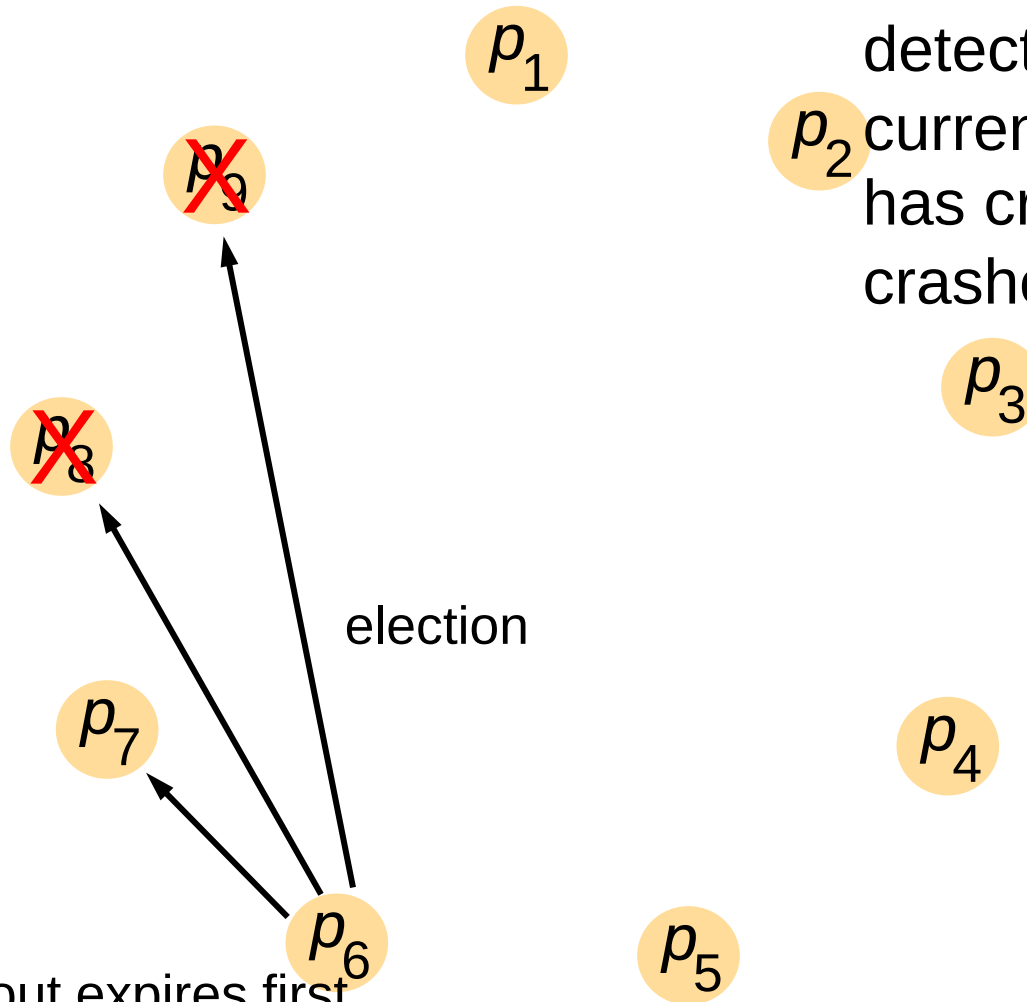
on receiving answer messages,
 p_7 sets a timeout for the
coordinator message

Example of Bully Algorithm



Example of Bully Algorithm

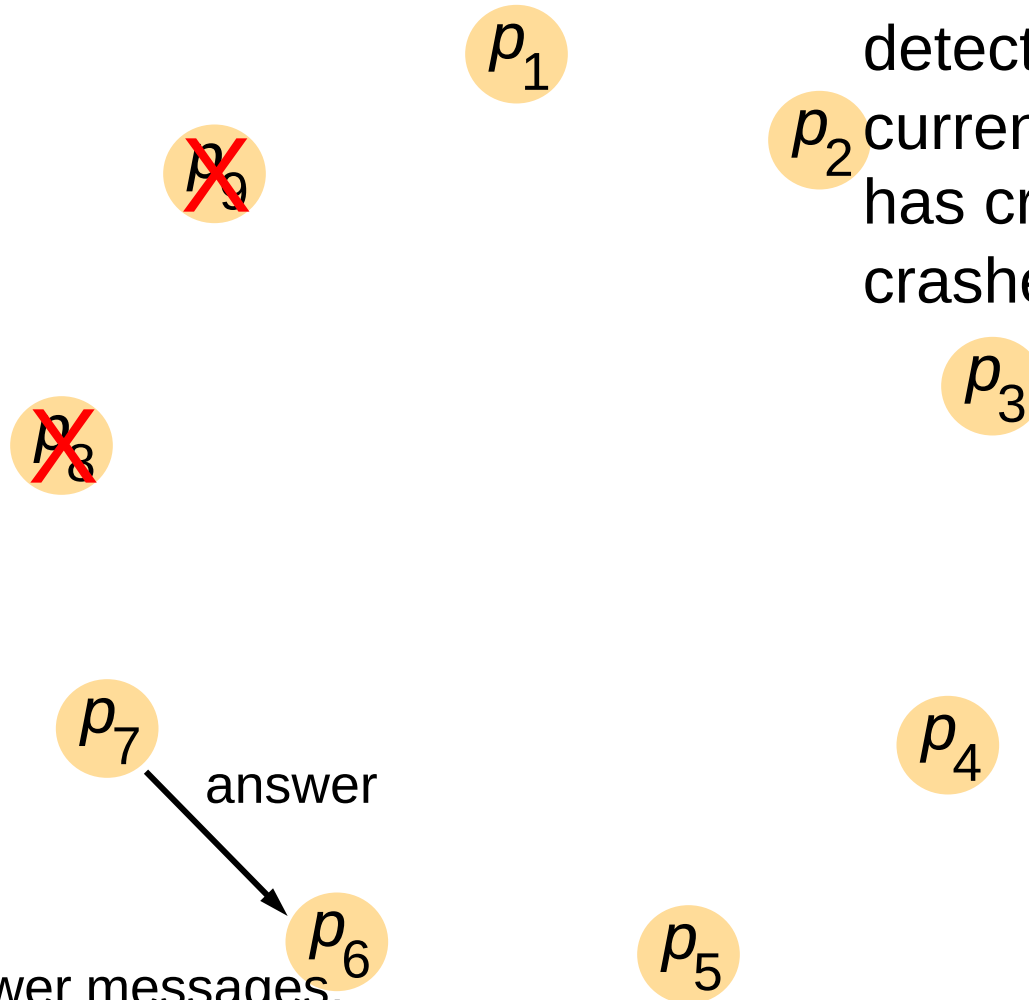
Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



assume p_6 's timeout expires first,
 p_6 begins another election and sets a timeout T for answer messages

Example of Bully Algorithm

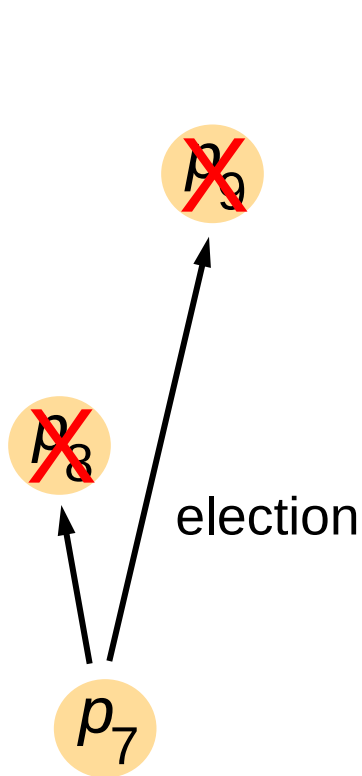
Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



on receiving answer messages,
 p_6 sets a timeout for the coordinator message

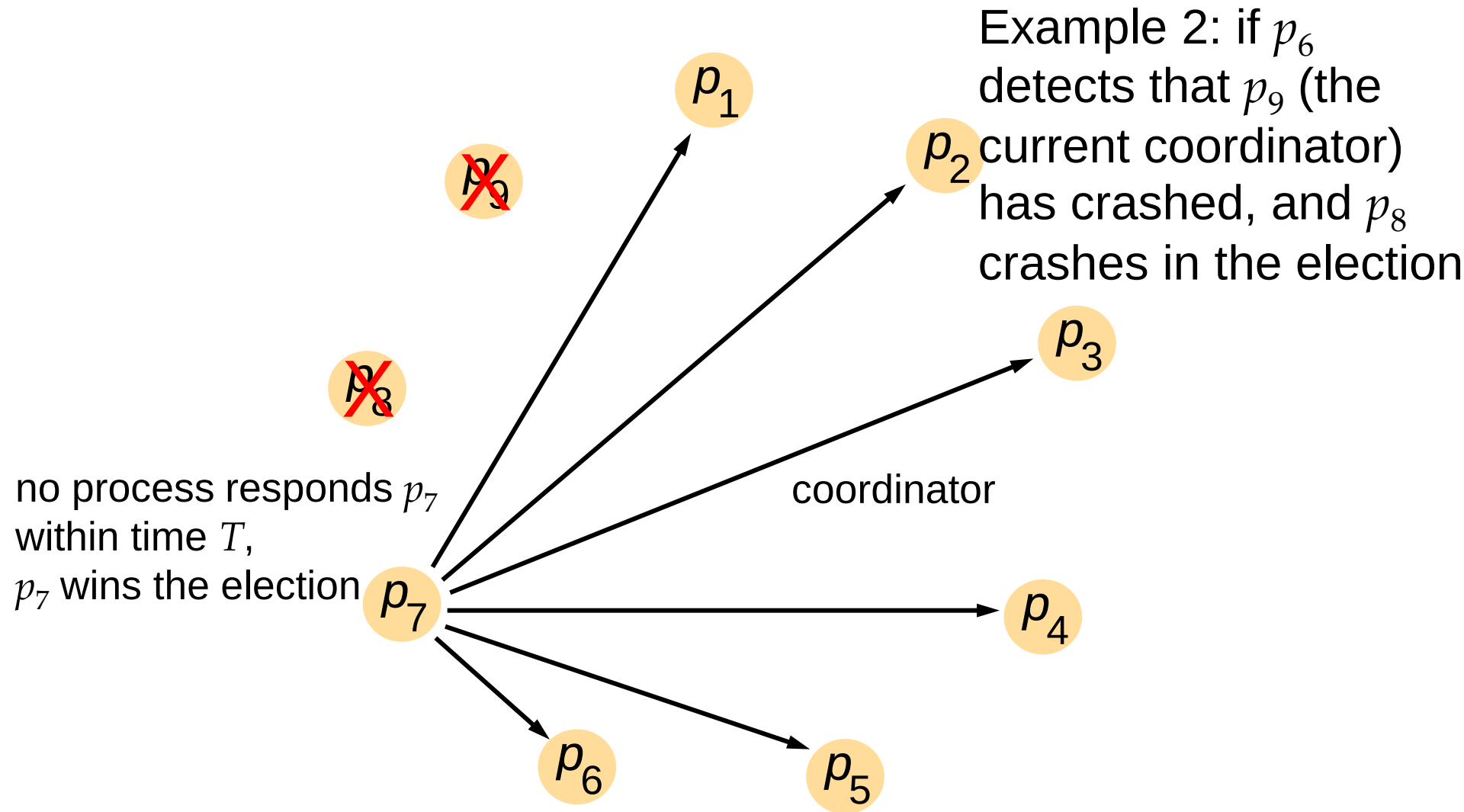
Example of Bully Algorithm

Example 2: if p_6 detects that p_9 (the current coordinator) has crashed, and p_8 crashes in the election



p_7 begins an election since it is not holding an election; p_7 also sets a timeout T for answer messages

Example of Bully Algorithm



Analysis of Bully Algorithm

- Correctness

- For any two processes, the one with smaller identifier will discover that the other exists and defer to it
- $\rightarrow$ it is impossible for two processes to decide that they are both coordinators

Analysis of Bully Algorithm

- Bandwidth consumption
 - Assume the current coordinator (the process with the largest identifier) crashes and no more process crashes during the election
 - Best case: the process with the second largest identifier detects the crash → 1 election message + $(N-2)$ coordinator messages
 - Worse case: the process with the smallest identifier detects the crash, it sends $O(N)$ election messages, and then each of the other processes that are alive begins an election → $O(N^2)$ messages
- Turnaround time
 - Best case: T + one message transmission

Outline

- Distributed Mutual Exclusion
- Election
- Consensus Problem

Consensus Problem

- Objective: for processes to **agree on a value** even in the presence of failures after one or more processes have proposed the value
 - In a transaction to transfer funds from one account to another, the processes involved must consistently agree to perform the respective debit and credit
 - In mutual exclusion, the processes agree on who can enter the critical section
 - In election, the processes agree on who is the coordinator
 - So, consensus problem is a generalization of mutual exclusion, election and many other problems

System Model

- A collection of N processes $p_1, p_2, \dots, p_N$ that communicate by message passing
- Synchronous or asynchronous system
- Up to f of the N processes may fail
 - They may have crash failures or arbitrary failures

Consensus Problem

- Every process p_i begins in **the undecided state** and proposes a single value v_i
- Processes communicate with each other to exchange values
- Each process p_i then sets the value of a decision variable d_i and enters **the decided state** in which p_i no longer changes d_i

Requirements of Consensus Problem

- Termination

- Eventually, each correct process sets its decision variable

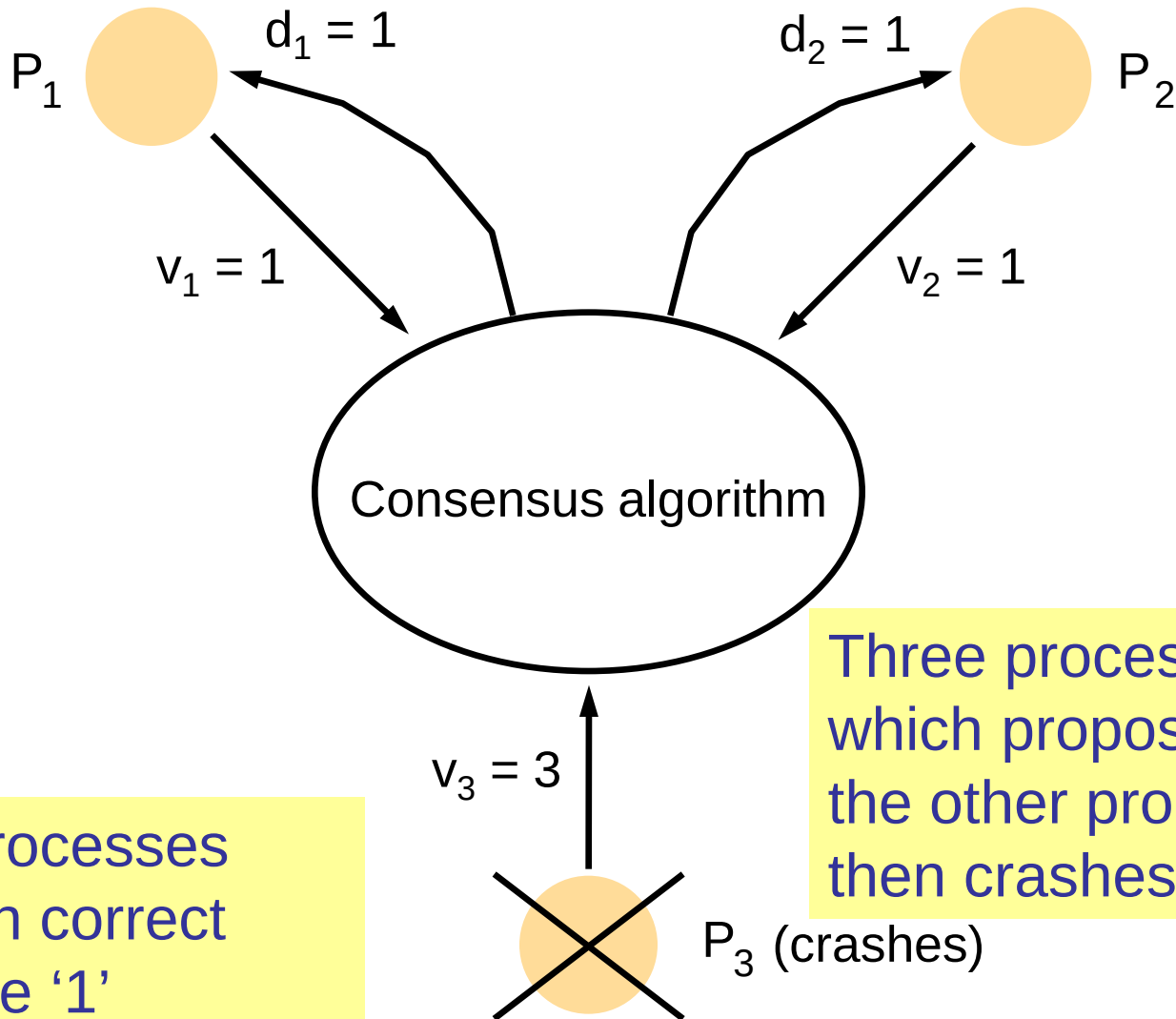
- Agreement

- The decision values of all correct processes are the same: for all i and j , if p_i and p_j are correct and have entered the decided state, then $d_i = d_j$

- Integrity

- If the correct processes all proposed the same value, then any correct process in the decided state has chosen that value

Example of Consensus Problem



The two processes that remain correct both decide '1'

Three processes, two of which propose '1', and the other proposes '3' then crashes

Consensus in Synchronous System

- Assumptions

- Synchronous system
- Up to f of the N processes may crash (no arbitrary failure)

- In the absence of process crash

- Each process p_i multicasts its proposed value v_i to **all other processes**
- Each process waits until it has collected all N values (including its own) and then **evaluates a function** to set its decision variable
 - e.g., $\text{majority}(v_1, v_2, \dots, v_N)$, or $\text{minimum}(v_1, v_2, \dots, v_N)$, or $\text{maximum}(v_1, v_2, \dots, v_N)$

Consensus in Synchronous System

- What is the problem when processes crash?
 - A faulty process may crash in the middle of multicast
→ it may not actually send values to all other processes
 - So, how to guarantee that processes receive the same set of values in the presence of crashes?
- Solution: an algorithm proceeds in $f + 1$ rounds
 - In each round, the correct processes multicast the values between themselves
 - At the end of the last round, all correct processes that survive must have received the same set of values
 - Then, each correct process evaluates a function to set its decision variable

Consensus in Synchronous System

Algorithm for process $p_i \in g$; algorithm proceeds in $f + 1$ rounds

On initialization

$Values_i^1 := \{v_i\}; Values_i^0 = \{\};$

In round r ($1 \leq r \leq f + 1$)

$B\text{-multicast}(g, Values_i^r - Values_i^{r-1});$ // Send only values that have not been sent

$Values_i^{r+1} := Values_i^r;$

while (in round r)

{

On B-deliver(V_j) from some p_j

$Values_i^{r+1} := Values_i^{r+1} \cup V_j;$

}

After $(f + 1)$ rounds

Assign $d_i = \text{minimum}(Values_i^{f+2});$

$Values_i^j$: the set of values maintained at process p_i before round j

Proof of Correctness

- Synchronous system $\rightarrow$ termination is obvious
- To prove agreement and integrity, we show that each process arrives at the same set of values at the end of the final round
 - Assume on the contrary that two processes differ in their final sets of values – a correct process p_i possesses a value v that another correct process p_j does not possess
 - p_i possesses v at the end of the last round $\rightarrow p_i$ cannot possess v before any round
 - because p_i is a correct process, if p_i possesses v in any preceding round, v would have been successfully sent from p_i to p_j

Proof of Correctness

■ Proof (cont'd)

- p_i possesses v at the end of the last round $\rightarrow$ at least a third process (other than p_i and p_j) must possess v before each round
- $\rightarrow$ let's say p_k possesses v before round r
- $\rightarrow$ similarly, p_k cannot possess v in any preceding round $\rightarrow p_k$ is attempting to multicast v in round r
- $\rightarrow p_k$ must have crashed in the middle of round r since otherwise, p_j would have received v from it
- So, in each round, at least a third process (other than p_i and p_j) is attempting to multicast v , and the process crashes in the middle of that round
- $\rightarrow$ at least $f + 1$ processes crash $\rightarrow$ contradiction

Consensus in Synchronous System

- Theoretical results: any algorithm to reach consensus despite up to f crash failures requires at least $f + 1$ rounds of message exchanges
- This lower bound also applies to the case of byzantine failures

Impossibility in Asynchronous System

- In the previous algorithm for synchronous systems, message exchanges take place in rounds
 - A process can be concluded to have crashed if no message is received from it within the round (setting the length of a round to be longer than the maximum delay)
- In asynchronous systems, processes can respond to messages at arbitrary times, so a crashed process is indistinguishable from a slow one
- Fischer proved that **no algorithm can guarantee to reach consensus in asynchronous systems even if there is only one faulty process**

Summary

- Distributed mutual exclusion
 - Central server algorithm
 - Ring-based algorithm
 - Ricart and Agrawala algorithm
- Election
 - Ring-based algorithm
 - Bully algorithm
- Consensus problem
 - Solution to consensus problem in synchronous system